

RE-RASSOR 2026

Design Document

Prepared by Group 19:

Kaitlyn Armstrong, Nam Dang, Toluwalope Elugbaju,

Max Grabov, & Lucas Marrou

Sponsored by the Florida Space Institute

Funded by the Stephen Hawking Center for Microgravity Research

University of Central Florida

Department of Computer Science – Senior Design 1 – Summer 2026

Submitted: August 2, 2026

Table of Contents

Table of Contents.....	1
Executive Summary.....	2
Introduction.....	3
What is RE-RASSOR?.....	4
Deliverables & Objectives	5
Minimum Viable Product	5
Detailed Technical Deliverables	6
Impact of Our Project	7
Possible Effects on Research.....	7
Possible Effects on General Populous	8
Other Uses of Our Rover and Arm Addition	9
Feasible Uses.....	9
Theoretical Uses.....	10
Implications & Ramifications Associated with Our Project.....	11
Legal Implications.....	11
Ethical Implications	12
Privacy & Security and Its Relation to this Project.....	13
Meet the Team	13
Kaitlyn Armstrong	14
Nam Dang.....	15
Toluwalope Elugbaju	16
Max Grabov	17
Lucas Marrou	18
Project History & Previous Teams' Productions	19
2022: The Previous Arm Attempt	19
2023: FSI's Current V1 Rover.....	21
2025: The Brushless DC (BLDC) Motor	23

2026 RE-RASSOR Final Design Document

Project Characterization	26
Our Commitment to Documentation	26
Our Tech Stack	27
Project Specific Materials	27
Concept of Operations	31
Design Documentation	35
Research & Investigation	35
Defining Requirements	36
Degrees of Freedom	37
Actuation	38
Final Design Direction	39
Our Inspiration During Initial Design	40
Testing Procedures	42
Minimal Viable Product Testing Procedure	42
General Complete Rover Testing	44
After Testing Analysis Procedure	47
General Mechanical Testing.....	47
General Software Testing.....	49
User Interface Testing	51
Final Design Testing	52
The Initial Design.....	53
The Mechanical Design.....	53
The Software Design.....	54
HTTP Requests	54
Integration of Previous UI Design.....	59
Alternatives & Considerations in the Initial Phase	62
Mounting Area	62
Materials.....	63
Mechanical Components	63

2026 RE-RASSOR Final Design Document

Management Considerations.....	65
Initial Feasibility Analysis.....	65
Initial Considerations of Stretch Goals & Feasibility	66
Autonomous Object Pickup	66
Additional Wrist Joint for Rotation	67
Interchangeable End Effector Attachments.....	67
Camera Integration into End Effector.....	68
Complex Counterweight System	69
User Interface Design Alterations for Precise Control	69
Strech Goal Prioritization	72
Suggested Long-Term Development Roadmap.....	73
For More Information on Our Considerations of Users During Our Design See the Administration Section of this Document.....	74
Design Diagrams & Descriptions	75
Use Case Diagram	75
Activity Diagram.....	76
Block Diagram	77
Component Modifications Made During Development	78
The Mounting Plate	78
The Arm Base	79
The Arm Base Pivot Connector.....	82
Planned Modifications to the User Interface	84
Conclusions	87
Where We Fell Short.....	87
Initial Timeline Issues	87
Administration	88
Budget & Material Procurement.....	88
Current Cost Breakdown	89
Arm Cost Table	89

2026 RE-RASSOR Final Design Document

Rover Body Cost Table	89
Printing Time Breakdown	90
User Stories	91
Rover Operator from a User Perspective	91
Rover Operator from a System Administrator Perspective	92
Future RE-RASSOR Development Team	92
FSI Sponsor	92
Development Team Member	93
Researcher	93
Summary of Current Completed Tasks and Subtasks	93
Jira Workflow	95
Acknowledgements	96
Project Guidance	97
Institutional Resources	97
Appendix	98
Previous RE-RASSOR Projects	98
Internal Documentation	98
Hardware Documentation	99
Software Documentation	99
Design Inspiration	100
Robotics & Technical References	100
Manufacturer Datasheets	101
Project Methodology	101

Executive Summary

The RE-RASSOR Arm Development project is a collaborative senior design effort sponsored by the Florida Space Institute (FSI) to expand the capabilities of an existing RE-RASSOR rover platform through the design, fabrication, and integration of a robotic manipulator. Previous iterations of the rover have successfully demonstrated autonomous mobility and excavation capabilities; however, the platform currently lacks a manipulator capable of interacting with its environment. The objective of this project is to develop a multi-degree-of-freedom robotic arm that can be mechanically integrated with the existing rover while utilizing the current software architecture and communication framework.

The project encompasses the complete engineering design process, including mechanical design, additive manufacturing, software development, electrical integration, and system testing. Mechanical components are being modeled using AutoCAD and fabricated through additive manufacturing to enable rapid prototyping and iterative refinement. Existing rover hardware and software are being leveraged whenever possible to minimize redevelopment while maintaining compatibility with the established control system. The software architecture will be extended through additional HTTP-based communication interfaces that allow independent control of each arm joint while preserving the modular design of the current rover application.

To reduce technical risk, the project follows an incremental development strategy in which individual subsystems are designed, validated, and integrated before full-system testing. Mechanical prototypes are evaluated for fit, structural integrity, and compatibility with the rover platform prior to final fabrication, while software components are developed and tested independently before hardware integration. This iterative approach enables design improvements to be incorporated throughout the project while reducing integration challenges during later development stages.

Upon completion, the robotic arm will provide the RE-RASSOR platform with enhanced manipulation capabilities, enabling future research into autonomous interaction with extraterrestrial environments. The final system is expected to serve as a modular platform that future senior design teams and researchers can continue to expand, supporting the Florida Space Institute's ongoing development of robotic systems for planetary exploration and other remote operation applications.

Introduction

What is RE-RASSOR?

The Regolith Excavation – RASSOR (RE-RASSOR) platform is a long-term multidisciplinary robotics project sponsored by the Florida Space Institute (FSI) and supported by the Stephen Hawking Center for Microgravity Research at the University of Central Florida. Rather than being a standalone senior design project, RE-RASSOR represents the continued development of an educational and research platform that has been expanded over multiple years by successive teams of students.

The rover is derived from NASA's RASSOR (Regolith Advanced Surface Systems Operations Robot) concept, which was developed to investigate autonomous excavation and material handling for future lunar and planetary exploration missions. While the original NASA platform was designed for large-scale excavation using dual counter-rotating bucket drums, the Florida Space Institute's implementation serves as a smaller, modular research platform that enables students to explore robotics, autonomous systems, software engineering, mechanical design, additive manufacturing, and systems integration within a realistic engineering environment.

Unlike projects that begin with a blank slate, RE-RASSOR follows a continuous development model. Each senior design team inherits the hardware, software, documentation, and lessons learned from previous teams before contributing to new capabilities for future development. This iterative approach mirrors many real-world engineering programs in which complex systems evolve through incremental improvements rather than complete redesigns. Consequently, maintaining compatibility, producing high-quality documentation, and designing modular systems are considered just as important as developing new functionality.

Over the past several years, previous teams have steadily expanded the platform's capabilities. Earlier teams established the rover's mechanical foundation, developed the software architecture, implemented ROS2 communication, created a web-based control interface, redesigned the drivetrain using brushless DC motors, and improved the manufacturability of the overall platform. These accomplishments provide the foundation upon which the current project is built.

The focus of the 2026 RE-RASSOR project is to extend the existing rover by designing, manufacturing, and integrating a robotic manipulator capable of interacting with objects in its environment. This robotic arm will enable the rover to grasp, transport, and accurately position

standardized 3D-printed pavers while remaining compatible with the existing mechanical structure and software architecture. Beyond providing new functionality, the project emphasizes modularity, maintainability, and comprehensive documentation so that future student teams and researchers can continue expanding the RE-RASSOR platform with minimal redevelopment effort.

Ultimately, RE-RASSOR serves two complementary purposes. As a research platform, it enables experimentation with robotic systems intended for remote and extraterrestrial environments. As an educational platform, it provides students with experience working on multidisciplinary engineering projects that integrate mechanical design, embedded systems, software development, additive manufacturing, systems testing, and long-term project maintenance. The work completed by each team becomes the foundation for the next, ensuring that the platform continues to mature through successive generations of student engineers.

Deliverables & Objectives

The main objective of our team is to design, build, test, and deliver a fully functioning, scaled robotic arm that integrates with the existing RE-RASSOR rover platform. As this project is built on multiple years of earlier student development, our baseline objectives are heavily weighted toward developing a solid, physically reproducible mechanical design while building out the web-based software framework to drive the arm.

Minimum Viable Product

To fulfill our contract, the completed system must achieve the following milestones:

Physical Manipulation Capabilities

The robotic arm must be mechanically capable of autonomously or manually targeting, lifting, moving, and precisely positioning the project's standard 3D printed pavers. These pavers weigh under one pound with diverse shapes and sizes. The arm must execute these tasks without causing structural deformation to itself or the rover's chassis.

Rover Stability Maintenance

An important performance objective is that the arm must operate within its specified envelope without causing the RE-RASSOR rover to flip, tip, or lose traction. Because the base body of the rover (excluding wheels, lid, and electronics) has a relatively low mass because it is primarily constructed from

2026 RE-RASSOR Final Design Document

lightweight PETG. Any extension of the arm shifts the center of gravity drastically. The arm's weight and motion profile must be balanced to maintain steady operations.

Control System Integration:

The rover control system must be easy to set up, visualize, understand, and use. And must incorporate arm control systems together seamlessly with the rover's base control system without causing system to back up or reduce performance.

Detailed Technical Deliverables

When the project is finished, the team will give the sponsor a complete handover package. This package, which is divided into four different categories, guarantees complete continuity for upcoming student teams:

Computer Aided Design (CAD) Packages

Fully Realized 3D Assembly Models: Complete. STEP and .STL files detailing every link, joint, bracket, and gear assembly of the robotic arm.

Rover Modification Models: CAD files detailing the exact geometry of any custom supplemental mounting plates or counterweight brackets affixed to the rover body.

Software and Firmware Source Code

Web Interface Upgrades: Clean frontend code integrated into the existing web application UI that features intuitive controls for managing the arm's multi axis movements seamlessly alongside the primary drive controls.

HTTP Endpoint Registry: Well documented backend application routes managing incoming HTTP requests assigned to arm actuation.

Low Level Motor Control Logic: Optimized C/C++ Arduino sketches managing the distribution of pulse width modulation (PWM) or serial commands to the custom Brushless DC Motors (BLDCM) and tracking rotary feedback. All of the source codes will be securely tracked across the designated GitHub branches.

Fabricated Physical Hardware

Functional Arm Assembly: A fully manufactured 3D printed mechanical arm primarily composed of PETG plastic to provide structural endurance against environmental heat loads.

2026 RE-RASSOR Final Design Document

Electromechanical System: An integrated suite consisting of lightweight Brushless DC Motors, associated gear reduction units (such as cycloidal or worm gearing), complete power wiring harnesses, and local network microcontrollers.

Engineering Documentations

Assembly Instructions: Step by step documentation detailing physical assembly, fastener sizing, wire routing constraints, and maintenance requirements.

Verification and Validation Reports: Documented logs of unit tests, full system integration checks, and formal acceptance testing witnessed by the project sponsor.

Future Work Backlog: A clear record of failed design iterations, lessons learned, and recommended improvement tracks for subsequent teams.

Impact of Our Project

Possible Effects on Research

A working rover arm would give future teams a usable starting point for experiments. They could study how the arm behaves on a light mobile base, then compare later changes against the same setup. This matters because each new team currently spends part of its schedule learning the inherited rover and rebuilding missing knowledge. A complete arm, supported by clear files and test records, would move that work forward. Later groups could begin with a known assembly and focus on a specific question such as joint accuracy, payload limits, or rover stability during movement.

The arm also creates a useful test case for the brushless DC motors developed for earlier RE-RASSOR work. Those motors were designed for the rover drivetrain, where the load stays close to the chassis. The arm places them in a more demanding arrangement. A motor at the shoulder must move the links and every component mounted past that joint. Measurements from loaded lifts would show how much gear reduction is needed and how the motors respond after repeated cycles. Results from these tests could guide later motor selection without forcing each team to repeat the same basic evaluation.

Additive manufacturing gives the project another research value. Printed arm parts can be adjusted quickly after a fit issue or structural weakness is found. Each revision can be tied to a specific observation from testing. A team might change the wall thickness after a mount flexes, then compare the revised part during the same lift. Keeping the failed part and recording the conditions that caused the

problem would produce evidence that later teams can use. Over time, the project could build a practical record of how printed components behave in a small robotic manipulator.

Operation of the arm may also support research on user control. The rover already uses a remote interface, and the new controls will place driving and manipulation in the same system. A new operator can reveal problems that the development team may overlook. Hesitation during a pickup task may point to an unclear label or an awkward control layout. Camera feedback could be studied in the same way once a view near the gripper is available. These observations would help future teams improve the operator experience without changing the pickup task used for evaluation.

Possible Effects on General Populous

The project is most likely to affect the public through education. RE-RASSOR gives students a physical example of code controlling a machine. A command on the screen causes a motor to move, which then changes the position of the arm. This connection is easy to see during a demonstration and can make robotics feel less abstract. The paver task is also simple enough for an audience to follow without a long technical explanation. Students can watch the rover approach an object, pick it up, then place it somewhere else.

Demonstrations can also show how engineering work develops. Finished products often hide the failed prints and revisions that came before them. RE-RASSOR offers a chance to show those steps. A part may fit poorly during the first print, or a joint may loosen after repeated movement. The team then measures the problem and changes the design. Showing that process gives students a more accurate view of engineering. It also makes failure easier to understand as useful information is gathered when the team records what happens. This information can then be used during the next revision.

The rover may help generate interest in space related robotics because its task can be connected to work performed in remote environments. Paver handling can represent a small version of moving construction material or positioning equipment. Its purpose is to make engineering questions visible. Students can see why weight matters when the arm extends and why communication problems become serious once software controls moving hardware.

Schools and robotics clubs could use the design after the project files are organized for reuse. A group with access to a suitable printer may build selected components or create a smaller classroom version. They may also use the existing arm as a base for a project focused on one attachment. This lowers the amount of work needed for a short project. The usefulness of the files will depend on the instructions that accompany them.

2026 RE-RASSOR Final Design Document

Access will still vary between schools. Some programs may lack a printer that can produce larger parts. Some programs may also lack a mentor who is comfortable working with motors and batteries. The documentation should state the equipment required and explain which tasks need close supervision. Simplified activities could help groups that cannot build the complete rover. A student might study the gripper on a fixed stand or use the CAD files to design a compatible attachment. These smaller uses would still support the educational goal of the project.

Other Uses of Our Rover and Arm Addition

The arm gives RE-RASSOR the ability to interact with objects around the rover. Paver pickup is the required demonstration, though the same movement can support several classroom activities. One example is a sorting course. Printed objects could be placed in marked areas, and the operator would move each one to a matching location. The task could begin with manual control. A future team could use the same course after adding visual markers or a simple detection program.

A camera near the gripper would allow the arm to serve as a mobile inspection tool. The operator could position the camera close to a test part or a location that is difficult to view from the main rover camera. This use places little demand on the gripper because the arm would mainly hold the camera in position. It could be useful during troubleshooting as well. A close view of a joint or printed component may help the team observe movement that is difficult to see from outside the test area.

The end of the arm could support tools made for specific lessons. A scoop would allow small amounts of simulated regolith to be moved during a demonstration. A probe could touch marked points as part of a positioning exercise. A tray could carry a light object across a short course. Each tool would need to stay within the tested load of the arm. A standard attachment point would make these projects easier because students could work on the tool without changing the rest of the mechanism.

Feasible Uses

Manual paver manipulation is the clearest use supported by the current design. The operator would drive the rover into position through the existing interface. Arm controls would then move the gripper toward the paver. A successful sequence ends when the paver is lifted and placed at the target. This task gives the team a direct way to evaluate the design because every subsystem must work together. The rover must remain stable during the lift, and the gripper must hold the object until release is commanded.

The rover can also support supervised robotics lessons using the software path already planned for the arm. Students can observe how an input moves through the application and reaches the motor

controller. Logs can be used to show what happens when a request is accepted or rejected. This gives the demonstration of a software component without requiring students to operate the physical rover immediately. Once the command path is understood, they can connect it to the motion they see during a controlled test.

A documented end effector connection is practical within the current project. The gripper already needs a secure connection to the wrist. Defining that connection with fixed measurements would allow later teams to make new tools that fit the same arm. The mechanism should remain simple enough to print and assemble with common hardware. A screw-based plate or keyed connection could meet that goal. Testing would confirm that the attachment stays rigid during a paver lift.

Camera assisted operation is also realistic if the first version only provides a live view. A small camera placed near the end effector could help the operator line up the gripper with the paver. The video would also make failed pickup attempts easier to study because the team could see the contact point. The camera mount would need to stay clear of the fingers and the ground. Its wiring would also need enough movement to follow the arm without being pinched.

Stability testing may show a need for rear weight. A fixed counterweight or small storage area could be added with limited design work. The amount should come from testing with the arm extended. Too much weight could reduce mobility and add stress to the rover. A measured solution would provide rear support without making normal driving impractical. An active balancing system would require more development than the current stage allows.

Theoretical Uses

A more advanced version of the rover could support automated placement of standardized building pieces. The current pavers provide a small-scale example of that type of work. A future system would need to detect each piece and determine where it should be placed. It would also need dependable rover positioning, so the arm begins each movement from a known location. The present project cannot provide that level of accuracy, though it can establish the mounting and control structure that later work would need.

Regolith handling is another possible direction for the arm. A scoop or sample container could allow a future rover to collect material from a test area. The material could then be moved to a storage location or delivered to another device. Real field use would place much greater stress on the joints than the current paver task. Dust would also affect bearings and printed gearboxes. Work in this area would require protective changes and stronger validation under simulated environmental conditions.

Several rovers could eventually cooperate on a larger task. One rover might carry material. A second rover could handle placement. Their software would need a shared understanding of location and task status. The rovers would also need a way to recover when one unit stops or loses communication. This direction could turn RE-RASSOR into a platform for studying coordination between mobile robots. It would require a dependable single rover before multi rover behavior could be evaluated in a useful way.

Remote maintenance could become possible with a more precise arm and a specialized tool. A future rover might press a switch or reposition a small instrument. Some tasks could require the arm to connect to a cable, remove a cover, or other tasks that require precision. These actions require accurate depth information and much tighter control than paver pickup. Force feedback would also be needed to prevent damage when the tool contacts equipment. The current modular end effector goal could give later teams a place to begin experiments in this area.

The rover could also carry sensors into locations that are difficult to inspect from a fixed camera. The arm could position a depth sensor close to the surface. The rover could remain at a safe distance during the measurement. A future system might recognize an area of interest and move the sensor into place automatically. This would combine mobility with controlled sensor placement. The arm would need better position feedback before measurements could be tied to a precise location.

These uses describe possible development paths. The current prototype is only an early step toward those capabilities. Their purpose in this document is to show how decisions made now may affect later work. A rigid wrist interface and organized control code would make expansion easier. Poorly documented dimensions or unclear software dependencies would make the same ideas much harder to pursue.

Implications & Ramifications Associated with Our Project

Legal Implications

The RE-RASSOR arm is being developed as a university teaching and research prototype. Its legal concerns are limited at this stage, though the team still needs to handle reused work correctly. The project draws from earlier RE-RASSOR repositories and from the EEZYbotARM design. Credits and license notices should remain with any files that require them. The final repository should also make clear which parts were created by our team and which parts were adapted from earlier work.

Publicly available designs may still include conditions for reuse. A file may allow modification and still require attribution. It may also restrict how a copy is distributed. Future teams should be able to find this information without searching through old repositories. Our documentation should identify the

source of major reused components and preserve links to the original work. This protects the contributors and reduces confusion if the project is shared.

Liability becomes more relevant during demonstrations. The arm has moving joints that can create pinch points. An extended arm could cause the rover to tip. Operations should remain supervised and should stay within the limits established during testing. The team should provide an inspection process for the person running the demonstration. Warnings should also be included for any area where a hand could come into contact with the mechanism.

Privacy concerns are limited in the current design because a camera is not required for the baseline arm. Camera integration may be explored later as a stretch goal to improve operator visibility or support object detection. Any future implementation should avoid unnecessary recording and follow UCF or host organization policies when video is saved.

Laboratory and event policies will continue to apply to the rover. Battery handling must follow the rules of the facility where the system is tested. The same is true for printer use and electrical changes. A future commercial version would require a much broader review of product safety and data practices. The current team should describe only the standards and limits that have been evaluated.

Ethical Implications

Our main ethical responsibility is the safety of anyone near the rover. The arm is small enough for a classroom lab, yet an unexpected movement can still catch a finger near a joint. Testing should begin with individual components before the complete arm is powered. A fixed stand gives the team a safer place to identify binding or uncontrolled motion. Full rover tests should begin only after the stop command and emergency shutdown have been verified.

The team should describe the system honestly. The project draws from ideas used in planetary robotics. The prototype itself remains a supervised classroom platform. A successful paver lift should be reported as one defined manipulation task completed under known conditions. Clear limits make the project more credible and help future students understand what still needs to be solved.

The educational goal also requires a simple and intuitive control mechanism and detailed documentation that is understandable to people outside the team. Internal names may make sense during development but confuse a new operator. Labels should describe the physical movement they cause. Assembly instructions should explain each step without assuming that the reader attended our meetings. This helps future teams use the system safely and gives more students a reasonable chance to learn from the project.

2026 RE-RASSOR Final Design Document

We also have a responsibility to preserve the work that came before us. Earlier teams produced code and mechanical ideas that made our project possible. Their contributions should remain credited in the final files. Our own failed parts should be documented with the same care as the successful versions. Deleting that history would make later teams repeat problems that we have already observed.

Autonomous movement would create additional ethical concerns because the rover would begin making decisions from sensor input. A detection error could send the arm toward the wrong object. Future automated modes should keep a human operator ready to stop the system. Motion limits should be conservative until repeatability has been demonstrated. The camera should focus on the task area and collect only the information needed for operation.

Material use is part of responsible development as well. Some failed prints are expected during prototyping. Careful measurements and fit checks can keep the amount reasonable. Reusable fasteners should be recovered when a design is retired. Slicer settings that produce a strong part should be recorded so later teams can begin with a proven setup. These habits support the teaching purpose of the project because they show that engineering decisions affect resources as well as performance.

Privacy & Security and Its Relation to this Project

Privacy and security are secondary concerns for the current RE-RASSOR arm because it is a supervised teaching system on a local network. The rover handles control commands and camera data, not sensitive personal records. The more immediate risk is physical movement caused by an incorrect command. Basic security still matters because any device connected to the rover can potentially influence that movement. The level of protection should match the classroom setting and the actual risk of the prototype.

The rover should operate through the dedicated router used by the project. Access should be limited to approved devices during testing. The router should use a changed password, and the setup instructions should tell future teams where that password belongs without placing it in the public repository. The server should validate every arm request before it reaches the motor controller. Joint values outside the tested range should be rejected. Loss of communication should also cause motion to stop.

If recording capability is added later, any saved footage should have a clear purpose tied to testing or documentation. The files should be stored in an approved location with access limited to the project team. Project repositories also need basic security. Passwords and private keys should stay out of GitHub. Local network addresses should remain in setup notes. Example configuration values can show future teams how the system works without exposing active credentials.

Meet the Team

This section contains statements from our team collected at the beginning of this project. Each member summarized their previous experiences, reasons for ranking this project highly in the assignment process, personal motivations for the project, and hopes for what could be accomplished

Kaitlyn Armstrong

I am excited to be working on the RE-RASSOR Arm Development project because it combines several areas that have interested me for many years, including robotics, software development, engineering design, and project leadership.

My interest in robotics began in high school through the FAU Robotics League and the FIRST Robotics Competition (FRC), where I gained experience working on multidisciplinary engineering teams and developing solutions under strict deadlines. During that time, I not only worked on the hardware and software of metal channel robots but also served as Lead Programmer and Co-President of my high school's team. These experiences exposed me to both the technical and organizational aspects of robotics projects and taught me the importance of communication, planning, testing, and documentation when working on complex systems.

Beyond the technical experience I gained through robotics, I am also excited about the educational impact of projects like RE-RASSOR. My own interest in engineering and computer science began in middle school through LEGO Robotics, which introduced me to programming and problem-solving in a hands-on environment. That experience ultimately played a significant role in my decision to pursue a degree in Computer Science. Because of that, I appreciate projects that can inspire future students in the same way and want to provide other young individuals with avenues to inspiration like what LEGO and high school clubs provided for me. One aspect of the RE-RASSOR project that I find particularly rewarding is its potential to serve as both an engineering platform and an educational tool that can encourage younger students to explore STEM fields.

One of the aspects of this project that interests me most is the opportunity to build upon an existing engineering platform rather than starting from scratch. The RE-RASSOR project has been developed by multiple teams over several years, which creates unique challenges involving system integration, documentation, and long-term maintainability. This is especially so given the number of repositories both private and public, and the number of different universities associated with this rover. I find these challenges particularly interesting because they reflect many of the problems encountered in real-world engineering projects.

2026 RE-RASSOR Final Design Document

From a technical perspective, I am interested in learning CAD and 3D printing to add to my skill repertoire. While I have previous robotics experience, much of it focused on programming and traditional robotics construction methods, making this project an excellent opportunity to broaden my skill set. Furthermore, I am interested in improving project documentation and knowledge transfer throughout the development process. During our first sponsor meeting, I noticed documentation was mentioned as a weakness of earlier teams, and I believe this is a problem that can be improved through diligent work and planning. Effective documentation is crucial for any long-term project, and I look forward to helping improve this area.

Overall, I see this project as an opportunity to apply my background in robotics, software development and computer science while gaining experience in new technical areas and contributing to a system that can continue to be improved by future student teams.

Nam Dang

I am excited to be a part of the RE-RASSOR development team because it combines several areas that I have always been passionate about: space, robotics, and programming. I have always been fascinated by the beauty and mystery of space and the incredible engineering that allows humans to explore beyond Earth. The creativity involved in designing robotic systems checked out all that I am interested in.

Since I was young, I have always been curious about how things work. I often took apart household devices and toys simply to understand their mechanisms and learn how they all work together. That curiosity and interest then turn into an interest in programming and engineering, where I can now not only understand how systems work, but also contribute to creating and controlling them myself. The idea of developing a rover capable of navigating terrain and manipulating objects through a robotic arm is exciting because it combines software, electronics, and mechanical design into a single project.

Another thing that drew me to this project is that RE-RASSOR offers complete freedom in the design process. Although earlier senior design teams have already developed a working rover, I still have the option to either build upon that work by integrating a robotic arm or redesigning the system entirely, and the arm can be whatever I want; it just needs to work. Which encourages creativity, sparks discussion, and is simply more fun than following a predetermined design. And although I have not yet had the opportunity to use a 3D printer myself, I have always wanted to design and make something of my own.

2026 RE-RASSOR Final Design Document

While I have previously built simple robots using motors and mechanical components, my experience has primarily been on the hardware side, with only some limited opportunities to work with the programming and control aspects of robotics. This project presents an ideal opportunity for me to expand my knowledge in those areas and gain experience developing a complete robotic system as well as creating meaningful connections with people and peers who share the same interests.

Coming from a developing country, I did not have the same level of exposure to robotics, and programming opportunities as some of my peers. However, this has only made me more eager to learn and take advantage of this opportunity. I am genuinely excited for the chance to contribute to RE-RASSOR and learn from my teammates.

Toluwalope Elugbaju

I ranked RE-RASSOR highly because it was one of the few projects that involved robotics and gave us the opportunity to work on a physical system. Most of my coursework has been focused on software, so I wanted to be part of a project where I could apply those skills while also learning about mechanical design and system integration.

Another reason I was interested in RE-RASSOR was the open design process. The robotic arm does not already have one fixed solution that we are expected to copy. Our team has to make design decisions based on what the rover can realistically support and what the arm is expected to do. It forces us to think through the reasoning behind each decision instead of only completing a set of assigned steps.

I am also motivated by the responsibility that comes with building on an existing rover platform. Since RE-RASSOR has been passed between multiple teams, our design choices have to make sense beyond the first working prototype. The arm has to be understandable enough for future teams to modify or expand without starting over. That makes the project more meaningful to me because the final result is not only the arm, but also the process and documentation that allow the project to continue.

Another part of the project that interests me is its connection to education. RE-RASSOR is meant to be an accessible robotics platform that can help students learn about robotics and engineering design. That gives the project a purpose that isn't just building a prototype. If the arm is designed and documented well, future students could use it as a starting point for learning or for developing new features. Because of that, proper documentation and maintainability will be an important part of the project.

2026 RE-RASSOR Final Design Document

I am also interested in the possibility of future expansion. One idea our team has already discussed is using interchangeable end effectors so the rover could perform different tasks with the same general arm structure. Even if that becomes a stretch goal, it's still useful to think about how the design could be expanded later. Designing with future work in mind makes the project more meaningful because the team is contributing to a platform that other groups will continue building on.

Overall, I am motivated by the opportunity to help develop a functional robotic arm and see the design progress from early discussion into a working prototype. I am expecting the project to be challenging because the arm has to work as part of the overall rover. It needs to perform its task while still being practical for future teams to understand, repair, and build on top of. Since RE-RASSOR is also meant to be used in educational settings, it's important to make the final design clear enough for students to learn from it as well. That is what makes the project interesting to me. The work must result in a functioning system but also must support the goal of making robotics more accessible to students.

Max Grabov

RE-RASSOR is a project that touches a lot of areas I am currently interested in, namely robotics, and systems programming. As someone who has been doing systems programming in the past year, I am excited to have the opportunity to work on a final project that reflects my interests.

My experience with robotics goes back to middle school, when I joined my school's robotics team for the First Tech Challenge (FTC), and then later First Robotics Competition (FRC) in high school. Even after graduating, I remained involved by returning as a mentor, helping students develop programming skills and problem-solving abilities.

Through both my experience as a student and a mentor, I've had the opportunity to work both with software and hardware on projects that share many similarities with RE-RASSOR, which is why I believe that I have the experience necessary to succeed on this team. While I haven't directly worked on the same tech stack as RE-RASSOR, I'm confident that I can learn new tools quickly and adapt to an unfamiliar technical environment. I view working on RE-RASSOR as an opportunity to contribute to a project that interests me, and deepen my expertise in a field that I'm invested in.

To go in depth about the technical outcomes, I am excited to practice and learn more about CAD, electronics, and other hardware. Most of my robotics experience has been on the software side, and while I have experience working with other microcontrollers and electronics, it is not near my experience in software. I believe working on RE-RASSOR will deepen my skills in these areas, as the project requires

all components of robotics to be successful. Additionally, RE-RASSOR is building on top of an existing teams' work and decisions. While personal projects provide the opportunity to build systems from the ground up, learning how to maintain and improve an established codebase is an essential skill any software engineer must learn, and is something I am eager to gain more experience in.

Another reason I am interested in working on RE-RASSOR is the community impact. As someone who has been mentoring robotics for the past 2 years, RE-RASSOR's goals of being student friendly and deepening STEM involvement in young audiences resonate with me. One thing I would like to propose in the future is bringing the RE-RASSOR rover over to my school and starting a program there. I believe that it will be a big success over there due to the existing STEM support offered by the school. Having the opportunity to not only develop the project but also introduce it into other communities is an exciting prospect I would love to explore.

Lucas Marrou

I ranked the RE-RASSOR project highly because it involves topics I am interested in. Specifically, I'm interested in space exploration, robotics, and to a lesser extent 3D printing, all of which are part of this project. I also believe my skills are a good fit for this project, so I will have an overall smoother experience here than on a less fitting project.

I've long liked space exploration, in part due to how beautiful and interesting many of the phenomena out there are, both to look at and to learn about. For example, many nebulae have amazing shapes and colors, though some require specific imaging techniques to be seen with the human eye. There are also neutron stars, which while we cannot directly look at them, have interesting structures on a microscopic scale ("nuclear lasagna").

I've also always liked robotics. I've been interested in programming for most of my life, having started learning programming when I was only six years old, and robots are the most visible way to see programming having direct results in the physical world. In addition, I am interested in robotics due to how fun it can be to imagine and brainstorm ideas for robots, rovers, or similar machines, regardless of whether those designs will ever be physically realizable.

3D printing is somewhat interesting to me due to its unique applications as a way to quickly and automatically create arbitrary shapes. It is especially useful as a complement to robotics and exploration, as it can be used to make a variety of specialized machines quickly, without needing molds for each, requiring the designs to use the same set of parts, or requiring additional parts to be sourced manually.

2026 RE-RASSOR Final Design Document

My skillset matches the listed recommendations for this project well, as follows:

- **Programming Experience:** I started programming when I was only six years old and have been interested in programming ever since. I have experience with C, C++, Python, Java, Assembly, as well as various esoteric or uncommon programming languages.
- **CAD Experience:** I have been certified in Autodesk Inventor, a relative of the common CAD program AutoCAD, but more specialized for complex 3D engineering tasks. I also have some experience with TinkerCAD, having used it as part of a 3D printing seminar recently.
- **Interest/Experience in robotics systems:** I have previously been part of a team for the FIRST Robotics Competition, back in 2020. However, I was unable to ever show off my work, as the competition was cancelled due to the Covid-19 pandemic AFTER my team had already arrived to compete.

I was also specifically interested in this project due to the association with NASA's Kennedy Space Center. I've visited the Kennedy Space Center before, and I am interested in potentially working there one day. My hope is that by being part of this project, I may have an opportunity to prove my skills to them and improve my chances of being accepted to work there.

Project History & Previous Teams' Productions

The work this team produced would be nearly impossible without the efforts of our predecessor teams. The section below summarizes their efforts and how they impacted our design and its early considerations during the design phase.

2022: The Previous Arm Attempt

Project Accomplishments:

The 2022 RE-RASSOR team was the first team tasked with creating a robotic arm to manipulate 3D printed pavers for the EZ-RASSOR rover. Like our own team, they were tasked with creating a low cost, almost entirely 3D printed arm to be mounted onto the rover to complete various tasks. This team focused on the software side and developed a complete software architecture around the robotic arm. They implemented a ROS-based control framework, with an in-depth simulation, and investigated computer vision techniques using NVIDIA Jetson Nano and cameras. Similar to ours, they also created an application to control the arm through the Raspberry Pi & Arduino. While this project has a larger scope

than ours and was ambitious and successful in what they were trying to achieve in the software side, their physical design had significant mechanical challenges that caused the arm to be unable to integrate into the RE-RASSOR platform.

One of the primary limitations that the 2022 team had was the significant emphasis on the software portion of the project, rather than the mechanical design and robustness needed to manufacture a mechanical arm. While their simulation showed good results, the translation to a real design proved to be much more difficult.



Figure 1: [1] 2022 Preliminary Simulation

Limitations:

The biggest issue this team faced, and one that heavily influences our current design, is the weight of their proposed arm. The arm relied on several stepper motors distributed across the joints of the arm to control it. As mentioned in the section about the 2025 RE-RASSOR team, stepper motors are very powerful but much heavier than the BLDC motor design. As the arm extended outward, each joint was forced to support not only its payload but also the weight of every motor and structural component farther along the chain. Since the motors were so heavy, there were significantly higher torque requirements at the base joints of the arm, which reduced the actual lifting capability. This is something the team ends up identifying in their final design document.

The team also experienced difficulties transitioning from software simulations to hardware implementations. The team mentions that while they were successful in simulating the arm's motion, integrating the hardware, electronics, and controllers introduced further challenges that didn't have enough time for a thorough implementation. They recognize that if they had validated their hardware design early on before focusing on the simulations and complex software, this may have prevented many of these integration problems for them.

Lessons Applied to our Design:

The work done by the 2022 RE-RASSOR team provided us with valuable lessons that directly influenced some of our design decisions for our arm. One of those decisions was to keep our arm as light as possible using the BLDC motors from the 2025 team. Since the weight of the arm required not only the base but the joints as well to produce high torque, we wanted to avoid this scenario if possible. Learning from the 2022 design, we plan to use a stepper motor for the base of the arm connecting to the chassis and using BLDC motors for the joints of the arm. This way, we can still use the power of the stepper motor at the base to rotate the whole weight of the arm, and it keeps the weight on the low side.

Another design decision that we are going with is to keep the 3D print infill percentage on the low side. When looking at the arm design from 2022, one thing that stood out to us was the infill on the arm pieces, as they were very high percentages and some cases 100%. This indicated to us that we want to avoid using such a high infill for arm pieces and keep heavier pieces for the base of the arm. In a similar way to using the BLDC motors instead of stepper motors for the joints, this allows us to keep the base strong but the arm itself light.

Finally, their team showed us the importance of focusing on the hardware of the project, instead of diving headfirst into the software of our mechanical arm. While their software was extremely impressive, especially their simulations for their whole entire bot, it led to a lot of time being spent on the software, when some of it could've been spent on validating the hardware design. Learning from their experience, our team decided to dedicate two members of our team to working on our hardware design, with our PM assisting us until we are confident in the arm design and it has been tested. This way, we can have software and hardware design happening at the same time and produce prototypes as soon as possible to allow software development to begin.

In summary, while the original RE-RASSOR arm did not fully achieve its intended goal, it established an invaluable technical foundation for our team. By analyzing both its successes and its limitations, our team has been able to develop a more practical mechanical design centered on lightweight construction, brushless DC motors, and ensuring that we spend a significant amount of time vetting our design.

2023: FSI's Current V1 Rover

Project Accomplishments:

The 2023 RE-RASSOR Extension team focused on expanding the software of the existing RE-RASSOR arm, rather than redesigning the mechanical structure. From their GitHub repository, they primarily focused on integrating modern ROS2 into the RE-RASSOR software infrastructure, and designing the remote-control interface, a mobile app, used to control the rover.

The largest accomplishment in software development they did for the RE-RASSOR is to refactor using ROS2 Humble. This modernized and improved the earlier software architecture for the rover itself. In addition, they also created updated ROS packages, scripts to launch the rover software and systemctl services for the rover. These improvements created a more maintainable software platform for future development.

The next accomplishment was the creation of a React Native application to control the RE-RASSOR rover. Previously, the team at FSI had to use a mobile app, which had issues due to apple requiring constant updates to stay on the app store. This currently controls the rover and has the opportunity to swap into different control modes, which is perfect for our project. Their modular design allows us to easily add new commands to the web application without significant architectural changes to the whole application.

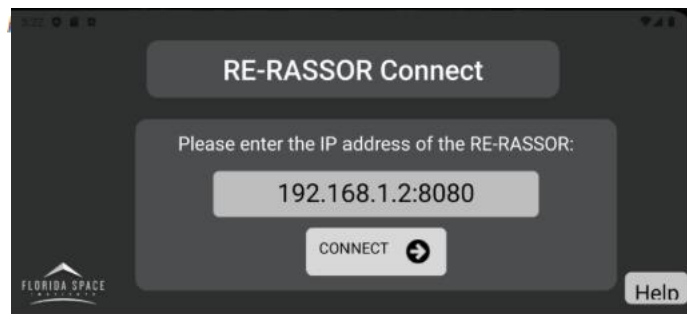


Figure 2: [5] 2023 Team's Rover Connection UI

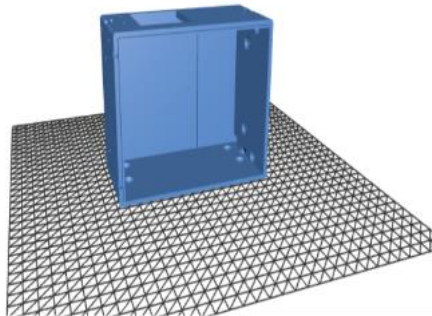


Figure 3: [2],[4] Current RE-RASSOR Body in CAD Software

Mechanically, the team worked to bring the RE-RASSOR rover into its current state. This includes all the current CAD models on the RE-RASSOR rover, as well as updating the old EZ-RASSOR

rover with a new shoulder design. Because this team mostly worked to improve the software on the rover, most of the borrowed and referenced work will pertain to their software.

Effects for our Project:

This team's work directly influences most of the software development we have to do for the arm design. Since this team set up the entire HTTP request system through the localhost application, our team is deciding to build that by adding an arm module for our robotic arm. This will be an additional page reusing and repurposing existing code for the base of the rover. Similarly, the ROS2 communication framework can be extended to support our new BLDC motor controllers we plan to use from the 2025 RE-RASSOR team. While the underlying hardware differs between the two, using the 2023 team's work with ROS2 and the rest of the software on the rover, we will just need to add new modules dedicated for arm control.

In relation to our stretch goals, the extension team also implemented the groundwork for a camera subsystem on the RE-RASSOR rover. What they wrote enables camera streaming from the rover that can detect ArUco markers in the lab. This is great for our design, as we can use their camera module to add functionality to move the arm to a certain offset from these markers and/or other movements. From their work, we don't need to focus on writing a camera system from the ground up and instead can immediately start leveraging a native library for RE-RASSOR.

2025: The Brushless DC (BLDC) Motor

Project Accomplishments:

The 2025 RE-RASSOR BLDC motor team successfully redesigned the drivetrain system of the Florida Space Institute's RE-RASSOR rover by replacing the previous stepper motor design with a well-suited brushless DC motor system. The primary objective of this was to decrease the rover's weight and improve energy efficiency while maintaining the original rover's architecture. This caused the new design to focus on constant rotational control rather than power as with a stepper motor. Since our project relies on us using their BLDC motor design for our arm, this assists us greatly with the nature of building a mechanical arm requiring precision over power.

Beyond a prototype, the 2025 RE-RASSOR team produced extension documentation including CAD files for all components, large documentation regarding process, materials, and project history, allowing us to adapt their work to our own seamlessly.

Mechanical Design & Process:

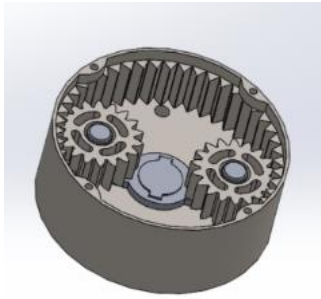


Figure 4: [1][5] CAD RE-RASSOR Gearbox

While the primary focus for the 2025 RE-RASSOR team was creating a new motor for the rover to use, and controlling it, the team also accomplished significant mechanical integration with the current RE-RASSOR design. Instead of redesigning the whole entire rover to fit their needs, they took a modular approach and designed the BLDC motors to be compatible with the current chassis and other 3D printed parts. This is due to the project being intended for additive manufacturing, allowing students or researchers to make additions to the rover without friction.

Their design process involved evaluating motor mounting methods, configuration of their gearbox, and alignment of the shaft of the motor with the rest of the mechanical components. The team compared a direct-drive and geared-drive solution for their gearbox before recommending planetary gearboxes or multi-stage spur gear reductions where additional torque was required. This means that their design can be adapted to scenarios where more torque is required, or resources are scarce, and the cost must be cheap, allowing students and teams like ours to adapt their design to our needs.

Another important contribution was maintaining compatibility with the rover's existing 3D printed chassis. Rather than redesigning the entire vehicle, the team modified only the parts necessary to accommodate the lighter BLDC motors. This reduced manufacturing costs and ensured that previously printed rover components could still be reused. In addition, the team worked to ensure that materials needed for constructing the gearboxes and motors themselves used readily available materials at the FSI lab and already being used in other places of the rover. This makes it much easier and cheaper to manufacture the rover, and any additional add-ons we need.

Software Design & Development:

The software developed focused on supporting the mechanical design goal of implementing the BLDC motor design, which is allowing for efficient rotational control of the motors, while maintaining

compatibility with the rest of the RE-RASSOR rover's architecture. To accomplish this, the team continued the use of the Arduino environment's C++ library to get low level access to the hardware.

The software is designed to interface with the main controller for the rover, which then passes commands to the Arduino Mega, which processes and generates PWM signals to control the motor. A major design goal was maintaining constant motor speed despite changing mechanical loads. To accomplish this, the software supports sensor feedback from Hall sensors or rotary encoders, allowing the controller to continuously monitor motor position and speed. This feedback enables more stable motor performance under varying operating conditions.

Due to the communication protocol mirroring the original stepper motor interface, the high-level rotor software did not require major refactoring to accommodate the new drive system, which reduces the effort to integrate the BLDC motors, and will allow for future projects utilizing their work to require few code changes.

Effects for our Project:

Due to the nature of our group's work, the effort made by the 2025 team will greatly influence our hardware and software development and design. Since the goal of our project is to design a robotic arm that *must* use this team's BLDC motor design, their work directly ties into what we need to do. From the creation of their design, it drives us to utilize their motors in our design due to their lightweight design, and strong control of the motor's rotation. Since a mechanical arm must be lightweight and stable, the BLDC motor created by the 2025 RE-RASSOR team gives us an immediate solution to two core challenges we would have to face. Without their design, we would either use the previous stepper motor design and integrate it into our arm, leading to heavy weight and too much power, or create an entirely new motor and gearbox design, significantly increasing the scope of our project. Therefore, we can focus on the actual architecture of the arm and use their gearboxes and BLDC motors when needed. As shown later in the document, there are a few key places that they will be placed to control the joints on the arm, and since they are already integrated with the rest of the rover, it is easy to build an arm that utilizes both the BLDC motors and a stepper motor for the base.

Their discussion and research into gearbox selection is valuable to us because robotic arms require high joint torque with precise control. Although our application emphasizes positional accuracy rather than continuous vehicle motion, the tradeoffs between direct drive, planetary gearboxes, and spur

gear reductions remain nearly identical to the 2025 team's study. Their analysis provides useful guidance when determining the appropriate transmission for each arm joint.

In terms of their software driving our design, our team will be interfacing with their code to control the BLDC motors over HTTP requests like the drive controls on the rover. Since their software is modular and easy to use, will have their packages as a dependency and modify the rover Arduino code to communicate with our arm subsystem. This allows us to focus on a high-level software design, without needing to delve into the low-level details involved in programming the motors from scratch.

Overall, they establish a strong baseline for our design for which our team can start our robotic arm, without worrying about hardware implementation. This way, we can focus entirely on the mechanical design and communication between the localhost server and the rover.

Project Characterization

Our Commitment to Documentation

Documentation is especially important for RE-RASSOR due to the way the project is passed between student teams. Our arm design cannot end as a prototype that only our group understands. Future teams should be able to open the project files, understand the current design, and continue development without reverse engineering from scratch. The documentation should make the final design understandable as a system.

The mechanical documentation should clearly describe the final arm assembly. This should include the CAD files, part versions, print material, print settings, and assembly steps. If a part has a specific orientation, support requirement, or tolerance issue, that should be written down near the part it applies to. Future teams should be able to tell which files current and which ones are only used for testing. File names should also be organized clearly, so the final design is not mixed with outdated prototypes.

Any part that changed during testing should be documented with a short explanation of what changed and why the previous version was rejected. If a joint, mount, or gripper part fails during testing, that failure should be recorded and explained instead of being removed from the project's history. Failed designs can still prevent future teams from repeating the same mistake. For example, if a joint was too loose, a mount flexed too much, or a gripper could not hold the paver consistently; those results should be included with the design notes.

Assembly documentation should be detailed enough for someone outside our group to rebuild the arm. The instructions should show how the printed parts connect, where each motor is mounted, how the gripper is installed, and how the arm attaches to the rover. Photos or diagrams should be included when written instructions are not enough. If a step requires careful alignment or a specific order of assembly, that should be stated clearly. This would reduce confusion for future teams and make repairs easier if the arm needs to be taken apart.

The software documentation should explain how the arm controls connect to the existing rover system. Each command should have a clear purpose and should match the physical movement it creates. Future teams should be able to identify which command controls the base, shoulder, elbow, wrist, or gripper without searching through the full codebase. Any setup steps, dependencies, or configuration details should also be written clearly, so the software can be run again after the project is handed off.

The control documentation should also connect the user interface to the physical arm. If a button, keyboard input, or HTTP request moves a joint, that relationship should be recorded. This will make debugging easier when a command does not produce the expected movement. It also helps future teams modify the interface without accidentally changing the wrong control behavior. If any commands are limited for safety or only meant for testing, those limits should be stated.

Testing documentation should connect design decisions to actual results. Each major test should record the setup, the part version used, the observed behavior, and the design change that followed. This applies to mechanical testing, software testing, user interface testing, and full rover integration testing. A short record of what worked and what failed will be more useful to future students than only showing the final successful demonstration.

The final handoff should also include known limitations. If the arm works only within a certain range, that range should be stated. If the gripper struggles with certain paver positions, that should be recorded. If the mount, wiring path, software controls, or counterweight concept still needs improvement, those issues should be left as clear notes for future work. Limitations should be written directly so future teams can plan around them instead of discovering them during testing.

The final folder should separate CAD files, software files, testing notes, assembly instructions, and reference material. A main README should explain where each part of the project is located and what files are needed to rebuild or run the system. This makes the project easier to build on and gives future teams a stronger starting point.

Our Tech Stack

A variety of physical materials, software, and other technological products are or will be used in the rover, the associated peripherals for the rover, and/or the development process as broken down below

Project Specific Materials

Printing Materials [1][5]

- Polylactic Acid (PLA)
 - Used for rapid prototyping and early design iterations.
- Polyethylene Terephthalate Glycol (PETG)
 - Used for manufacturing final robotic arm components intended for permanent installation on the rover.
- Bambu Lab X1 Carbon with Automatic Material System (AMS)
 - The primary 3D printer used to manufacture the robotic arm and its associate prototype components.

Mechanical Components [1][5]

Rover Electronic Components

- Brushless DC (BLDC) Motors
 - Actuate the rover wheels, robotic arm joints, and end effector through custom 3D-printed cycloidal gearboxes.
- Raspberry Pi 4 and/or Libre Computer (Le Potato)
 - Execute the rover control software and facilitate HTTP communication between the user interface and onboard systems.
- Arduino Mega 2560
 - Controls individual BLDC motors and interfaces with low-level hardware.
- Emergency Stop Switch
 - Model: CMORSUN BE102 Push Button

2026 RE-RASSOR Final Design Document

- Provides an immediate method of disconnecting power during testing or emergency situations.
- Battery Pack
 - Model: Nermak LiFePO₄ 12.8V Battery
 - Supplies electrical power to the rover and robotic arm.
- 12V Brushless Cooling Fan
 - Maintains airflow through the rover enclosure to cool onboard electronics.
- Camera System
 - Primary Model: ArduCam UC-A53
 - Provides live video feedback for teleoperation.
- Router
 - Model: NetGear Nighthawk
 - Creates an isolated wireless network for rover communications, avoiding restrictions imposed by institutional networks.
- Engineering Workstations
 - Used for CAD modeling, embedded software development, and user interface programming.

Software Components

Version Control

- Git & GitHub
 - Version control, collaboration, and source code management.
- Google Drive
 - A backup artifact for project handoff and presentation purposes
- Onshape

- A webapp CAD platform used by FSI for the purposes of sharing and managing finalized component and mechanism designs

RE-RASSOR Software [1][5]

- Arduino Firmware
- Microprocessor Software
 - Network communication services.

RE-RASSOR User Interface [1][5]

- React Native
- Expo
- HTTP Connection Service

Development Environments [1][5]

- Arduino IDE
- Android Studio
- Visual Studio Code

Simulation

- Gazebo
 - Simulates rover operation prior to physical deployment.

Computer-Aided Design (CAD) Software

- AutoCAD
- Fusion 360
 - Used throughout the mechanical design process.

Manufacturing Software [1][5]

- Bambu Studio
 - Generates G-code for the Bambu Lab X1 Carbon.

Communications and File Sharing

Discord

- Used for daily communication and project coordination.
- GitHub webhooks automatically post commits, pull requests, and repository activity to the team's Discord server, improving visibility into ongoing development.

Email

- Used for communication with the project sponsor and faculty.

Microsoft Office

- Used to prepare project documentation and deliverables.

Jira

- Used for project management through a Kanban workflow.

Testing and Development Locations

Florida Space Institute Laboratory

- Used for manufacturing, assembly, and robotic arm development.

Exolith Laboratory

- Used to evaluate rover and robotic arm performance in a simulated lunar regolith environment.

Concept of Operations

The primary operational objective of the 2026 iteration is the integration of a lightweight, multi axis articulating manipulator capable of acquiring, transporting, and precisely positioning standardized 3D-printed interlocking pavers without compromising the rover's chassis stability or mobility.

Operational Context & Mission Profile

2026 RE-RASSOR Final Design Document

In terrestrial and planetary exploration scenarios, surface mobility platforms must possess manipulation capabilities to accomplish infrastructure construction, sample collection, hazard mitigation, and instrument placement. The RE-RASSOR platform simulates lunar and Martian regolith excavation and construction workflows on a modular scale. Previous versions developed autonomous surface mobility mechanisms for the drum. The manipulator to be added in 2026 turns the rover into an active surface construction platform.

The mission profile is based on standardized paver handling. The rover must traverse simulated regolith terrain (e.g., UCF Exolith Laboratory), recognize standardized 3D-printed pavers weighing less than 1.0 lb (approx. 450 grams), approach the target within the arm's reach envelope, actuate the 4-DOF manipulator to grasp the paver, lift the payload while actively managing the vehicle's balance, traverse across the operational field, and place the paver into a designated construction grid or storage receptacle.

System Operational Scenarios & Lifecycle Phases

For safe, reproducible, and deterministic execution in field operations and educational demonstration, the operational workflow is divided into six distinct sequential phases. Each phase has explicit entry criteria, active execution monitoring, and exit condition checks.

Phase	Phase Name	Primary Objective	Key Operational Actions
Phase 0	Pre-Flight & Initialization	Verify power, network link, & hardware health.	Boot Raspberry Pi server, establish local Wi-Fi, run homing sequence on stepper base & BLDC joints, verify emergency stop loop.
Phase 1	Target Search & Approach	Navigate rover into payload reach envelope.	Operator drives rover using teleoperation UI and ArduCam feed; aligns chassis within

2026 RE-RASSOR Final Design Document

			15–25 cm reach vector of target paver.
Phase 2	Payload Acquisition	Grasp target paver securely with gripper.	Deploy arm from stowed position, compute joint/workspace target coordinates, actuate end effector gripper, verify contact load.
Phase 3	Transport & Stability	Traverse field while maintaining stability.	Retract arm into low movement carry configuration; adjust rear counterweight if necessary; drive rover to destination site.
Phase 4	Precision Placement	Deposit paver on target construction grid.	Extend arm to placement coordinate; actuate end effector release; execute brief clearance lift to avoid disturbing placed paver.
Phase 5	Stow & Power Down	Return arm to safe rest position and power off.	Return manipulator to stowed resting geometry on chassis plate; log operational metrics; issue server shutdown command.

User Operational Interactions

2026 RE-RASSOR Final Design Document

The concept of operations accounts for three distinct user roles interacting with the RE-RASSOR platform, each possessing specific operational requirements, interface privileges, and safety responsibilities:

Rover Field Operator:

This uses the React Native web/mobile control interface to teleoperate with the vehicle and robotic arm in real time. The operator relies on visual feedback from real-time telemetry gauges (joint angles, motor power states, battery voltage) and or from the ArduCam camera feed (additional goals). The operator maintains continuous situational awareness and is authorized to trigger manual emergency stop overrides.

Research & Academic Lead:

Configures automated test routines, conducts manipulation repeatability evaluations, and collects physical performance metrics (e.g., motor current draw under load, thermal dissipation, mechanical joint deflection). Interacts with high-level Python REST API endpoints and ROS2 telemetry topics to execute script-driven experiment suites.

System Maintainer / Future SD Team:

Modifies firmware logic, re-calibrates motor encoders, updates CAD structural components, and maintains codebase dependencies. Relies on comprehensive hardware wiring schematics, modular software documentation, and explicit mapping rules.

Environmental, Physical, & Electrical Constraints

Successful operation of the RE-RASSOR arm requires strict adherence to physical, thermal, electrical, and network boundary conditions:

Mass & Center of Gravity Envelope:

The primary chassis body is an additive manufactured using lightweight PETG. Uncompensated extension of the multi axis manipulator drastically shifts the system's Center of Gravity forward. To prevent tipping or loss of front-wheel traction, the arm structure must remain under 1.2 kg in total mass, with an operational payload limit of 1.0 lb. (450g) at a maximum forward reach of 30 cm.

Thermal Operating Limits:

BLDC cycloidal gearboxes and NEMA 17 stepper motors generate heat during sustained holding or lifting operations. PETG deforms at temperatures exceeding 80°C (176°F). Thermal management

protocols dictate continuous cooling airflow via onboard 12V fans and automated software power throttling if motor temp estimates exceed 65°C.

Dedicated Wireless Network Topology:

Institutional campus networks impose strict port security, blocking local peer-to-peer REST requests and SocketIO streams. Operations dictate using an isolated NetGear Nighthawk wireless router operating on standard 2.4GHz/5GHz bands with fixed IP assignments (e.g., 192.168.1.100 for the onboard microprocessor).

Power Delivery Architecture:

Electrical power is supplied with a 12.8V LiFePO4 battery pack equipped with an inline CMORSUN BE102 physical Emergency Stop switch. Step-down buck converters supply stable 5V power to the Raspberry Pi / Le Potato board and Arduino Mega 2560, while raw 12.8V bus power feeds the motor drivers.

End-to-End Operational Workflow Architecture

The complete operational data and physical actuation flow across the RE-RASSOR ecosystem follows a strict distributed pipeline:

1. *Input Capture:* The operator interacts with React Native UI controls (sliders, joysticks, or preset buttons) on a remote tablet or workstation.
2. *REST Network Dispatch:* The client app dispatches an asynchronous HTTP POST request containing structured JSON payloads to port 5000 of the Flask server (`server.py`).
3. *Verification & ROS2 Translation:* `request.py` validates payload schema constraints. Valid commands are instantiated as kinematic state objects in `command.py` and published to ROS2 topics (`/ezrassor/arm\_instructions`).
4. *Low-Level Serial Encoding:* The `serial\_driver.py` node converts ROS2 instructions into compact binary wire protocol opcodes and streams them across USB serial links (`/dev/arduino\_arm`) at 115200 baud.
5. *Microcontroller Execution:* The Arduino Mega 2560 receives opcodes, generates PWM signals, coordinates motor driver pulses, reads magnetic encoder feedback, and enforces hardware limit switches.
6. *Telemetry Feedback Loop:* Sensor data and ArduCam video streams return asynchronously to the Flask server via WebSockets (SocketIO) and are rendered on the operator dashboard at 30 FPS.

Design Documentation

Research & Investigation

Before any computer aided design began, our team performed a detailed investigation into existing manipulator designs, structural considerations, and previous work done by the 2022 arm team. While the primary objective was to design a robotic arm capable of manipulating the designated 3D printed pavers, we had to make significant engineering decisions before settling a design. This became very apparent when looking at the arm developed by the 2022 arm team. While their arm was strong enough to complete the task, due to the complexity and weight of the arm being too much, it was unable to even be attached to the RE-RASSOR rover. Weight, structural rigidity, manufacturability, software compatibility, power consumption, and future maintainability all influenced the direction of the project.

In addition, since our team was working off an existing system, we had to consider how the arm would integrate cleanly with the rest of the rover. This meant we did have some baselines we could immediately work off, such as what electrical components we would need (BLDC Motors being a requirement), and where the arm would be mounted. This, however, gave us some limitations, as we had existing hardware that we had to be mindful of (such as the center battery and the rest of the motor connectors). Throughout the investigation process, our team evaluated potential solutions against the project's minimum viable product while considering stretch goals that future development teams could pursue.

Defining Requirements

The first stage of investigation involved identifying the functional requirements of the arm we planned to build, before selecting any specific design. We decided to determine exactly what we expected the arm to do before modeling components in 3D Modeling software such as Fusion 360. The robotic arm needed to retrieve, transport, and accurately place standardized 3D-printed pavers while remaining mounted to the existing RE-RASSOR rover. Unlike industrial robotic manipulators that operate from rigidly mounted bases, the RE-RASSOR arm would function on a lightweight mobile platform whose stability changes continuously as the arm moves. This immediately introduced design constraints that heavily influenced every later engineering decision.

2026 RE-RASSOR Final Design Document

Due to these constraints, several requirements were immediately established for us:

- Sufficient arm reach to access 3D printed pavers in front of the RE-RASSOR rover.
- Adequate lifting capacity for the pavers.
- Low overall weight to minimize changes to the rover's center of gravity.
- Being compatible with the concept of additive manufacturing.
- Modular construction to allow for stretch goals and future development.
- Compatibility with the existing software and electronics architecture.
- Ease of maintenance for damaged components.
- Low mechanical complexity to be in line with the RE-RASSOR target audience, being school age children.

Through establishing these basic requirements as our baseline, designs could be evaluated against an objective list rather than intuition or feeling.

Degrees of Freedom

Selecting the appropriate number for our degrees of freedom was another critical and significant research effort for us. While additional joints can improve the dexterity of the robotic arm, they easily can multiply the mechanical complexity of the final product. In line with our weight and complexity requirement, we immediately knew that we would not be doing a robotic arm with 6+ degrees of freedom. While they do provide excellent dexterity and control, this would require more motors to pilot every joint, more filament and parts to print, significant engineering skill which we may not be equipped to handle due to our limited background knowledge and added weight. These considerations led us to believe we had to keep the Degrees of Freedom limited, while still having enough complexity to accomplish our basic requirements.

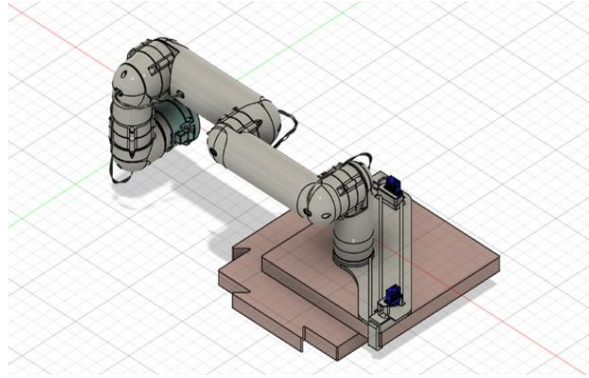


Figure 5: [1] Example 6+ DOF arm from the 2022 Team

On the same note, going for a simple 2 degree of freedom robotic arm would also not work in our favor. We had a reach requirement that we had to achieve, and having such a simple system would mean longer static parts which would worsen the weight requirements and be harder to control.

In the end, our team opted for a design consisting of a rotating base, shoulder joint, elbow joint, and end effector, totaling 4 degrees of freedom, with one of them being strictly limited to on the RE-RASSOR rover. This allowed us to have enough complexity to be able to manipulate our pavers in a simple way, while also ensuring that we remain in weight and have a properly scoped complexity. A Base degree of freedom helps us with this as well, since it is significantly easier to attach to the rover, and can be a stronger motor to provide enough torque to carry the entire weight of the arm. This design also permits us for out of scope stretch goals, for example, a wrist joint remains feasible as future work without requiring fundamental changes to the arm architecture, as parts can be swapped as we need.

Actuation

Motor selection proved to be one of the most influential decisions that we had to make in the investigation process. Depending on the actuator we choose, it can completely change the degrees of freedom, how we design certain components, weight limitations, and much more. In our project's case, our sponsor highly recommended and encouraged the use of BLDC motors. These were created by the 2025 RE-RASSOR team and proved to be a great solution to our needs. Before immediately choosing them, however, we had to weigh the pros and cons with what we could use.

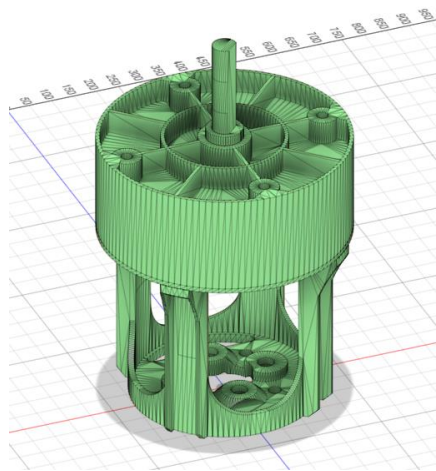


Figure 6: [5] Gearbox Reconstructed from the design made by the 2025 BLDC Motor Team

In our research, we came across two other options being stepper motors and servos. Traditional hobby servos were initially a great idea, because of their integrated position control and ease of implementation. However, commercially available servos capable of supporting the expected payload would significantly increase project cost while limiting scalability. While they would be great for a smaller scale robotic arm, they wouldn't work for our case. We also investigated stepper motors. We initially had much skepticism as these were the motors that were used for all the actuators of the 2022 RE-RASSOR arm team. While stepper motors are very powerful, this led to them having significant weight issues for other reasons. Since every motor mounted farther along the arm becomes additional payload for preceding joints, a fully stepper-driven arm would require dramatically larger motors near the shoulder and base. While we decided to not pursue using all stepper motors, they would be useful for our base joint since we needed a large amount of torque to control the entire arm.

Finally, Brushless DC motors presented a compelling alternative. Earlier RE-RASSOR development had already demonstrated a lightweight BLDC motor platform compatible with the rover's existing software architecture. Although BLDC motors require more sophisticated control electronics, they provide improved power density, lower weight, smoother operation, and greater efficiency than comparably sized stepper motors. Additionally, since the 2025 team made an easy to manufacture 3D printed solution for the gearbox, these were very easy to integrate with the rest of the RE-RASSOR rover. Additionally, since the part files were available to us already, we can simply assemble these components together to produce the gearbox produced in figure 6. Other options would require us to create or modify other gearboxes ourselves. This ended up saving us a lot of time when dimensioning the components we made, discussed later in the document.

Based on our research, our team decided to go with a hybrid approach. As mentioned earlier, we would control the base joint connected to the rover chassis with a stepper motor to get the maximum torque possible with little risk of weight issues. While for the rest of the arm, we would use BLDC motors to control the remaining joints. This way we can minimize the arm's weight while having sufficient power to control the mechanism.

One point that is up for debate is the end effector mechanism. Since the end effector is much more delicate, in testing we may find that using a BLDC motor for the end effector may be either too heavy, or too strong for it. In this case, we may opt to use a smaller servo or modify the existing BLDC motor gearbox to fit better with the end effector design.

Final Design Direction

Following our investigation and necessary requirements we derived, our team concluded that our approach would be a lightweight serial manipulator arm design designed for additive manufacturing. Similar to the 2022 team, which also opted for a serial design with chained stepper motors. The arm would use modular construction techniques, and incorporate a stepper driven base with BLDC motors driving the rest of the mechanism. Due to our limited engineering background, we also opted to look into existing designs online as a way to jumpstart our progress, at the recommendation of our sponsor. Rather than creating an entirely original mechanical layout, we elected to investigate existing open-source robotic manipulators that embodied many of these characteristics. Among the available designs, the EEZYbotARM provided the closest match to the project's objectives. Its modular construction, extensive use of additive manufacturing, and publicly available CAD models provided an excellent mechanical starting point that could be substantially modified to satisfy our requirements. While this did serve as a great starting point and inspiration, ultimately many of the parts done now, and in the future, are completely redesigned to fit our needs. This is due to the scaling of the EEZYbotARM, and different hardware that was used for their mechanisms. Despite this, it still served as a great reference for a working version of our final product. This investigation ultimately served as the foundation for the design adaptations discussed in the following section.

Our Inspiration During Initial Design

2026 RE-RASSOR Final Design Document

When researching mechanical arm designs, we took into consideration many factors that we needed to satisfy, such as degrees of freedom, weight, and overall control of the arm. One completely open-source design we found gives us a large foundation that we can work on to adapt to the RE-RASSOR rover. That design comes from the EEZYbotARM design. This project provides a fully open-source, desktop-scale robotic arm designed to be inexpensive, easy to manufacture, and almost entirely fabricated with common 3D printing materials. Like our initial sketches and ideas, this arm is a 3 degree of freedom arm (excluding the base), that utilizes modular and cheap components for their final product. Since all the CAD models are publicly available, it is a popular design for robotics projects to adapt an arm design to suit their needs.



Figure 7: [26][27] EEZYbotARM Sample Arm

One of the best parts of the EEZYbotARM is its emphasis on additive manufacturing. Nearly every structural component is designed to be 3D printed, including the base, shoulder, elbow, wrist supports, motor mounts, and linkages. This dramatically reduces manufacturing costs while allowing damaged parts to be quickly reprinted. The CAD models are organized into modular assemblies, making it straightforward to modify individual joints without redesigning the entire arm. The printed components are lightweight in their design as well. The arm also incorporates printed bearing supports and mounting surfaces that simplify assembly using commonly available hardware that is available in most labs.

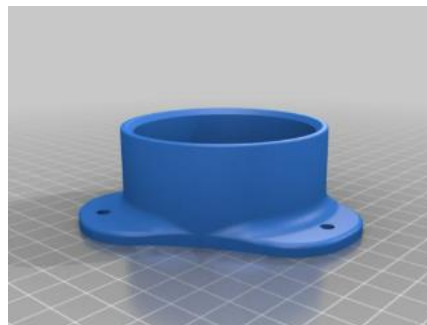


Figure 8: [26][27] EEZYbotARM Base

Effects on Our Design:

The EEZYbotARM serves as the mechanical foundation for our robotic arm but will require several significant modifications to satisfy the requirements of the RE-RASSOR project. The first core modification is completely overhauling the motor drive system to use the required BLDC Motors. This will help us in the end, as we would end up needing more strength compared to the small servos powering the arm design here due to the scaling. This will also require us to rewire and ensure that the EEZYbotARM design can interface well with the electrical components required to move each of the BLDC motors in our arm. In addition to using these motors to drive the arm, we are deciding to go with a stepper motor to power the base of the arm. This is because the EEZYbotARM modifications we will make will cause the arm to be much larger in scale, so we will need a strong base to generate enough torque for the arm.

Another major modification we will need to make is the scaling and reworking of many of the components in the arm design. Because this arm was developed as a standalone system, it is not meant to interface with other pieces of hardware and/or a chassis like the RE-RASSOR rover. Not only will we need to integrate a modified version of the arm into the front of our rover, but we will also need to scale the arm's design. Our application requires a larger workspace and increased reach, so many of the structural components will need to be resized.

A final effect on our design is the reinforcement of a claw/gripper design for the end effector. In our original sketches and talks, we wanted to go with a claw that could pick up the 3D pavers as specified. In seeing this design, it reinforced the idea that it is feasible and acted as proof of concept for other items. In the EEZYbotARM, the gripper is designed horizontally to pick up small items such as a ball. A new end effector will need to be developed to better accommodate the objects and tasks expected during operation (such as our pavers) while maintaining compatibility with the wrist assembly.

Overall, while many significant design changes are needed, the EEZYbotARM project acts as a great starting point for us as all the CAD files are publicly available, and we can see proof of concept of what we need to build for the rover.

Testing Procedures

Minimal Viable Product Testing Procedure

Testing Considerations

As the RE-RASSOR 2026 team has been recruited to a robotics-based project, we acknowledge that much of our work falls under the following three rules of robotics proposed by Isaac Asimov. Although fictional, Asimov's Three Laws of Robotics emphasize the importance of prioritizing human safety. Our testing philosophy similarly places personnel safety above all other project objectives.

- “A robot may not injure a human being or, through inaction, allow a human being to come to harm.” [28]
- “A robot must obey orders given it by human beings except where such orders would conflict with the First Law.” [28]
- “A robot must protect its own existence as long as such protection does not conflict with the First or Second Laws.” [28]

As a resulting, during the development of our project, we have and will continue to take meticulous care to test any products we produce not only to ensure our own safety during testing and manufacturing, but also for others who may later operate and edit the deliverables we produce as outlined above in our design approach and objectives. To achieve this, we as a team have developed and multi-avenue approach to safety including detailed documentation of tests, sectional component tests, complete rover tests, and fixed safety procedures while working on and testing the rover and its arm.

If any of the following conditions occur, the RE-RASSOR rover and robotic arm shall be shut down immediately. Testing will be suspended until the source of the issue has been identified, corrective actions have been implemented, and the affected testing phase has been successfully repeated. These criteria are intended to ensure the safety of personnel, protect project hardware, and maintain the integrity of testing results.

- Testing shall be paused immediately if any of the following conditions are observed:
- Visible cracking, layer separation, or structural failure in any 3D printed component.
- Audible cracking, popping, or excessive deformation of PETG, PLA, or other structural components during operation.
- Visible melting, softening, or heat-related deformation of printed components.
- Any fastener, bearing, shaft, joint, or structural component becoming loose or exhibiting excessive play during testing.
- Unexpected arm movement or motion outside of the commanded path.

2026 RE-RASSOR Final Design Document

- Excessive vibration, oscillation, or instability that could damage the rover or robotic arm.
- Impingement, pinching, stretching, or entanglement of wires, cables, or other electronic components.
- Loss of communication between the web application, Raspberry Pi, motor controllers, or robotic arm during operation.
- Failure of any emergency stop, return-to-rest procedure, or other implemented safety features.
- Failure of the end effector to securely retain or intentionally release a payload as commanded.
- Motor, motor controller, battery, or electronic component temperatures exceeding safe operating limits or showing signs of overheating.
- Smoke, unusual odors, electrical arcing, exposed conductors, or any indication of an electrical fault.
- The rover exhibits signs of tipping, excessive instability, or loss of balance during arm operation.
- Unexpected contact between the robotic arm and the rover chassis, electronics, testing equipment, or surrounding environment.
- Software behavior that prevents the operator from safely controlling the rover or arm.
- Any condition that presents a potential safety hazard to personnel or creates a reasonable risk of permanent damage to the rover, robotic arm, testing environment, or associated equipment.
- Following any test suspension, the team will perform a detailed post-test analysis to determine the root cause of the failure, document observations and corrective actions, and update the project's Kanban board as necessary. Testing will only resume after the issue has been resolved, and the affected testing phase has been repeated successfully.

While this is not a complete list of errors that could occur under testing, it is a comprehensive guideline for when a given test needs to be reevaluated to ensure a safe testing environment for all.

General Complete Rover Testing

The following seven step plan was devised to test all failure possibilities of the rover under the addition of our arm design. This seven-step system will be undertaken after initial individual mechanical and software testing to reduce chances of severe malfunction during the rover's operation.

Phase 0:

Prior to any powered testing, the mechanical frontend team will conduct a thorough inspection of the assembled arm and supporting hardware. This inspection will include verifying that all screws, bolts, nuts, spacers, and structural components are properly secured according to the

assembly documentation. Wiring will be inspected to ensure connectors are fully seated, cables are properly routed, and no wiring interferes with the mechanism's range of motion. Electronic components, including the BLDC motors, motor controllers, Raspberry Pi, and any auxiliary hardware, will also be inspected for proper installation.

The objective of this phase is to identify assembly defects before motion testing begins. Any issues identified during inspection will be corrected prior to proceeding to Phase 1.

Phase 1:

To isolate the arm from the rover and reduce the number of variables during initial testing, the arm and its mounting platform will first be assembled independently of the rover chassis. This allows the team to verify that the arm functions correctly before integration with the complete system.

Testing will begin by verifying the operation of the end effector through multiple open and close cycles. The operator will then test the arm's vertical motion by moving the arm from its fully lowered position to its maximum extension before returning it to its lowest position. This process will verify smooth motion, identify mechanical interference, and evaluate overall stability.

Following vertical motion testing, rotational movement will be evaluated by rotating the arm through its full range of motion at multiple extension heights, including both minimum and maximum reach. The operator will observe excessive vibration, structural flex, binding, or unexpected motor behavior.

The phase will conclude by returning the arm to its predefined resting position.

Phase 2:

Following successful motion testing, the team will evaluate all implemented safety features.

Testing will include verifying that the arm safely returns to its resting position following a simulated communication failure, confirming that the end effector maintains its commanded state during unexpected interruptions to prevent accidental release of payloads, and validating any software or hardware emergency stop procedures developed throughout the project.

Additional safety tests may be added as development progresses, and new safety features are incorporated into the system. Any safety feature that fails validation must be corrected before proceeding to payload testing.

Phase 3:

After successful completion of unloaded testing, the team will evaluate arm performance while manipulating project payloads.

Beginning from its resting position, the operator will command the arm to grasp a standard project paver using the end effector. The operator will then repeat the motion sequence established during Phase 1 while carrying the payload. This testing will evaluate joint stability, structural rigidity, motor performance, and end effector reliability under load.

The team will monitor for excessive deflection, reduced motion accuracy, motor overheating, slippage, or unintended payload release. Any deficiencies identified during loaded testing will be documented and incorporated into future design revisions.

Phase 4:

Once independent arm testing has been completed successfully, the arm will be mounted to the existing RE-RASSOR rover platform.

Initial testing will focus on verifying that arm operation does not interfere with the rover chassis, drivetrain, or existing electronics. The operator will perform basic arm movements while the rover remains stationary, including extending the arm, rotating through its operating range, and returning it to the resting position.

The team will also evaluate the structural integrity of the mounting system and monitor the rover for signs of excessive chassis flex or vibration resulting from arm movement.

Phase 5:

Following successful stationary testing, the rover will be operated while carrying the mounted arm.

The operator will drive the rover for predetermined distances while periodically stopping to extend and retract the arm at multiple positions. This testing will evaluate the effects of dynamic loading on rover stability, suspension behavior, and overall maneuverability.

Additional testing will include evaluating operation on uneven terrain and within simulated regolith environments to determine whether changes in the rover's center of gravity create instability or increase the likelihood of becoming immobilized. Results from this phase will guide any required balancing systems or counterweight designs.

Phase 6:

The final testing phase will combine rover mobility with full arm functionality.

Beginning with the arm in its resting position, the rover will navigate to a designated test paver before commanding the arm to retrieve the object. The rover will then complete a predefined driving course while carrying the payload before placing the paver at a target location.

This phase represents the complete intended operation of the system and evaluates the interaction between mechanical hardware, software, communications, and rover mobility under realistic operating conditions. Successful completion of this phase will demonstrate that the system satisfies the project's Minimum Viable Product requirements.

After Testing Analysis Procedure

Following the completion of each testing phase, the team will conduct a detailed inspection and analysis of the robotic arm, rover, and all supporting hardware before proceeding to the next stage of development. Rather than immediately implementing design changes, the team will first evaluate the results of testing to identify both successful design decisions and areas requiring improvement.

The mechanical frontend team will carefully inspect all structural components for signs of degradation or failure, including cracking, warping, layer separation, stress whitening, excessive wear, or deformation of PETG and PLA printed components. Fasteners, mounting hardware, bearings, shafts, gears, and joints will be examined for loosening, excessive play, binding, or other mechanical concerns that may have developed during testing. Motor mounts and structural interfaces will also be inspected to ensure that repeated operation has not introduced fatigue or unwanted movement.

The electrical and software systems will also be reviewed following each testing phase. Wiring will be inspected for signs of wear, connector loosening, or interference caused by arm movement. Motor temperatures, controller behavior, communication reliability, and software logs will be analyzed to identify unexpected behavior, missed commands, or potential reliability concerns.

Throughout the analysis process, the team will document all observations, failures, performance metrics, and recommended improvements. Each identified issue will be evaluated to determine its severity, root cause, and potential solution before being added to the project's Kanban board for future development. This analysis will directly guide design revisions and help determine whether the prototype is ready to advance to the next testing phase or requires additional redesign before further validation.

General Mechanical Testing

Mechanical testing will determine whether our arm can perform reliably before it is tested on the full rover. We will begin with individual printed parts, then move into joint assemblies, bench testing, and rover mounted testing. This order lets us isolate fit, alignment, and stability issues before full system testing adds more variables.

The EEZYbotARM model is being used as a conceptual reference for the general arm layout. It gives us a starting point for base rotation, linked arm sections, and gripper movement. The final RE-RASSOR arm still must be redesigned around the rover chassis, the required motor layout, the mounting space, the rover height, and the shape of the 3D printed pavers. The base is expected to use a stepper motor. The remaining joints are expected to use BLDC motors. That motor layout affects testing, since the base rotates the full arm structure while the upper joints handle lifting and placement.

We will first inspect the printed components for fitness and alignment. Each part should be compared against the CAD model before motors are attached. The arm depends heavily on printed geometry, so small errors can affect the assembly. A tight rotating joint can bind during movement. A loose mounting hole can create play once the arm is under load. A warped surface can throw off alignment across multiple parts. These problems should be corrected before powered testing begins.

Early prototypes should be printed in PLA when we are checking the form and assembly. PLA is easier to print and more practical for design changes. Once the geometry is more stable, PETG should be used for parts closer to the final design. This gives us a faster early testing process while still allowing load carrying parts to be tested with a more durable material.

Motor mounting should be checked before powered motion. The stepper motor at the base and the BLDC motors at the upper joints need to align correctly with the parts they move. Poor alignment can create friction and uneven loading. Before applying power, we should move each joint by hand through its expected range.

The base joint should be tested as its own assembly. It supports the rest of the arm and rotates the full structure, so looseness at the base will affect every movement above it. We should test for smooth rotation and consistent alignment after repeated movement. The mount also needs to spread load into the rover chassis without relying on major changes to the inherited platform. If the base shifts during testing, the upper arm results will not be reliable.

The shoulder joint should receive the most careful mechanical testing. When the arm lifts a paver, the shoulder joint carries the load from everything past that joint, including the upper arm assembly and

the object being lifted. The load becomes more demanding as the arm extends away from the rover's body. Testing should begin with unloaded motion then continue with the arm structure attached. A later test should use a paver or paver substitute held at different positions. During these tests we should watch for slipping, bending or loss of position.

The elbow joint controls much of the arm's reach. It may carry less total load than the shoulder, but it still affects end effector grabbing accuracy. If the elbow has too much play, the gripper may drift away from the intended position. If the elbow binds, the motor may struggle to move the arm smoothly. Repeated elbow movement should show whether the joint stays consistent or begins to loosen after use.

The wrist and gripper area should be tested separately before full arm testing. This is the part of the arm that directly interacts with the paver. A gripper can look correct in CAD and still fail if the fingers do not close evenly or if the contact surface cannot hold the paver securely. The gripper should open far enough, close around the paver, and release without catching. It should also be tested with the paver at slightly different angles to account for imperfect rover positioning during demonstrations.

Repeatability testing is also needed. The arm should lift a paver, hold it, move it, and release it through several cycles while we observe the joints and printed parts. Repeated operation can reveal loosened screws, gear wear, joint play, and motor heat buildup. These cycles will show whether the arm can perform the expected task without constant manual adjustment.

Rover stability testing should begin before the powered arm is fully integrated. We should mount the arm structure to the rover and manually place it in several positions. The test should start with the arm empty then repeat with a paver held at the end. If the rover becomes unstable when the arm is extended, we will need to change the mounting point, weight distribution, or counterweight plan. Previous RE-RASSOR arm attempts showed that stability can become a major failure point, so this issue should be tested before the final integration stage.

Before full rover testing, the complete arm should be tested on a fixed stand or bench setup. The arm should lift a paver, hold it in place, move it through the intended path, and release it without manual correction. Bench testing gives us a safer environment for identifying weak joints, poor alignment, unstable gripper behavior, and motor loading problems.

We will document each major test result alongside the part version, material, observed issue, and design change. Photos of failed prints, notes about joint movement, and explanations of rejected designs will make the project easier for future teams to continue.

General Software Testing

Software testing should begin before the arm is attached to the rover. The arm is expected to use the existing rover software structure, where commands are sent through the current network-based control system and integrated into the existing web application. Our first goal is to understand how existing rover commands move from the user interface to the rover controller. Once that path is clear, we can add arm commands in a way that matches the current system.

The first software task is to test the existing rover's HTTP requests. This will help us understand how commands are formatted, where they are processed, and how they produce physical movement. We will trace an existing drive command from the web interface to the hardware response, to model the arm commands after the same process.

Early arm command testing should happen without a physical arm connected. At this stage, we can use logging or simple test outputs to confirm that commands are being received correctly. A command intended for the shoulder should be recognized as a shoulder command. A command intended for the gripper should not trigger rover movement. This prevents more serious issues once motors are powered.

The software should also be tested against invalid input. During development, the back end might receive repeated commands or commands sent in an unexpected order. The software should reject or ignore bad input in a predictable way. It should also avoid uncontrolled movement if a command is repeated too quickly. Since the arm is a physical mechanism, software behavior must be considered when thinking of safety.

A stop command should be tested early. The operator needs a reliable way to stop the arm during testing and demonstrations. The stop behavior should be tested while the arm is unloaded, once the motors are connected, and later while the arm is holding a paver. We should also consider what happens if communication is interrupted. The arm should not continue moving indefinitely after the interface disconnects.

After HTTP request testing, the software should be tested with motors disconnected from the arm structure. This allows us to confirm that each command controls the correct motor and that each motor turns in the expected direction. If the shoulder command moves the wrong motor after the arm is assembled, it could damage the structure. Motor tests before testing a paver's load gives us a safer way to confirm wiring, motor assignment, and control logic.

The next stage should test the motors while connected to the arm on a stand. We should move one joint at a time and observe how the software behaves under real mechanical resistance. A motor might

respond correctly with no load then struggle once it must move a printed arm segment. These tests will show whether movement speed, command duration, or control behavior needs changes.

Software testing should also confirm that the original rover functions still work after arm controls are added. The rover must still drive, turn, and communicate after the arm software is introduced. After major software changes, we should retest basic rover movement before returning to arm testing. This prevents us from breaking the already integrated system.

The software should be documented as it is tested. Each arm command should have a clear purpose and a clear relationship to physical action. Future teams should be able to understand which command controls the base, shoulder, elbow, wrist, or gripper without reading through the entire codebase. Known limitations should be documented while they are still fresh.

User Interface Testing

User interface testing should focus on whether the arm can be operated clearly through the existing rover control environment. Since RE-RASSOR is meant for educational use, the controls should be understandable to people who did not design the system. The interface should let an operator control the arm without needing to understand the code for each command.

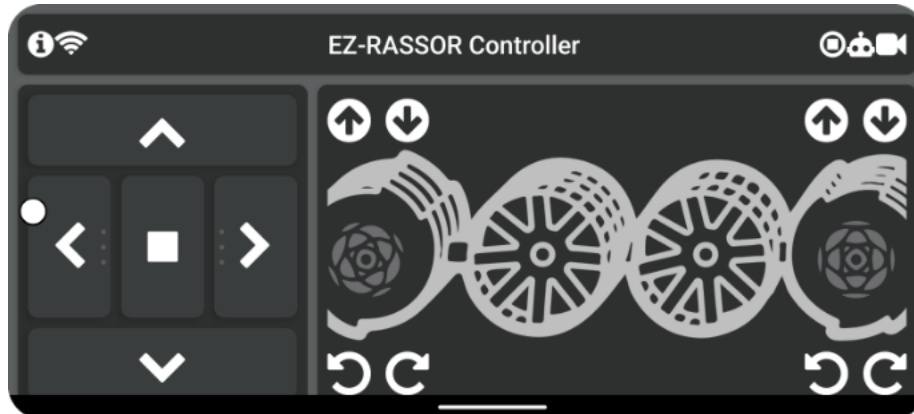


Figure 9: [5] Previous Driving UI Screen

The first UI tests should confirm that each visible control sends the correct command. This should be done before the arm is powered. If a button is labeled for the gripper, it should send a gripper command. If a control is assigned to the elbow, it should produce elbow movement once the hardware is connected. Logs can confirm the command behavior before the physical arm moves.

The interface should also be tested for clarity. Arm controls should be labeled in a way that matches the motion of the arm. Operators should not have to guess which control moves the shoulder, or

which one closes the gripper. If keyboard controls are used, they should be displayed in the interface or documented near the test setup. During a demonstration, the operator should be able to tell whether they are controlling the rover or the arm.

The layout should be checked during normal operation, not only during isolated command tests. Controls that look clear on the screen may still feel awkward once the operator must move between the base, arm joints, and gripper. The most important controls should be easy to find during paver pickup. Any stop or neutral control should also be placed where the operator can reach it quickly.

Timing should be tested as part of the interface. If the operator presses and releases a control, the arm should stop when expected. If a command moves the arm by a fixed step, the step should be consistent enough for the operator to predict. Poor timing can make the arm feel unsafe or difficult to use even when the mechanical design works.

The UI should also show communication feedback. If the rover is disconnected or the arm controller is unavailable, the operator should receive some indication that the command did not complete. A simple status message would make testing easier and reduce confusion during demonstrations. Feedback is especially useful during integration testing where a failed command could come from the interface, the network connection, or the hardware.

After the physical arm is connected, the interface should be tested during actual paver handling. The operator should use the UI to approach a paver, grip it, lift it, move it, release it, and return the arm to a resting position. This sequence will show whether the control layout works during real operation. A layout that seems reasonable during isolated joint testing might become awkward when many movements have to be coordinated.

UI testing should also include at least one person who did not build the interface. If another team member can understand the controls with minimal explanation, the layout is likely clear enough for demonstrations and future handoff. If they hesitate or press the wrong controls, the labels, layout, or documentation should be revised before final testing.

Final Design Testing

After all design revisions have been completed and the team is satisfied with the performance of the final prototype, a series of validation tests will be conducted to verify that the robotic arm consistently meets the project's Minimum Viable Product requirements. These tests are intended to evaluate the system under normal operating conditions as well as determine its practical operating limits.

The first validation will consist of a maximum payload test. The team will gradually increase the weight lifted by the arm until the predetermined design limit is reached. Throughout testing, the operator will monitor motor performance, structural deflection, joint stability, and overall rover balance to ensure that the system remains within safe operating limits. Any signs of excessive deformation, loss of stability, or unreliable operation will be documented and compared against the project's design requirements.

Following payload validation, the team will conduct a maximum runtime test. The arm will repeatedly perform its standard operating sequence over an extended period of time to evaluate long-term reliability. The objective of this test is to identify issues such as motor overheating, fastener loosening, excessive wear of moving components, degradation of 3D printed PETG components, communication failures, or software instability that may not appear during shorter testing sessions.

The team will then perform a maximum runtime under load test by repeating the same operating sequence while continuously manipulating project pavers. This testing represents a more realistic operating condition by combining extended runtime with continuous mechanical loading. Throughout the test, temperatures, structural integrity, repeatability, and overall system performance will be monitored to identify any reliability concerns before final project acceptance.

As a final validation and pending availability of Florida Space Institute facilities, the team intends to conduct testing within a simulated regolith environment similar to the intended operating conditions of the RE-RASSOR platform. Coordination for this testing will be conducted with the project sponsor to determine facility availability and scheduling. During this validation, the rover will repeat the previously established testing procedures, including mobility testing, arm operation, payload manipulation, and stability evaluation while operating on simulated lunar terrain. This environment will provide the most realistic assessment of system performance and help verify that the robotic arm integrates successfully with the existing rover platform under representative operating conditions.

Upon completion of all validation testing, the team will perform one final post-test inspection and document all performance metrics, observed wear, design limitations, and recommendations. These results will be incorporated into the project's final documentation and provided to future RE-RASSOR teams to support continued development of the platform.

The Initial Design

The Mechanical Design

Base Concept:

This preliminary concept illustrates the team's initial approach to integrating a robotic arm with the existing RE-RASSOR rover platform. Rather than designing an entirely new rover, the concept focuses on modifying the current platform while preserving as much of the existing drivetrain and software architecture as possible. The arm is intended to provide sufficient reach to manipulate the project's pavers while minimizing interference with the rover's existing systems approach to integrating a robotic arm with the existing.

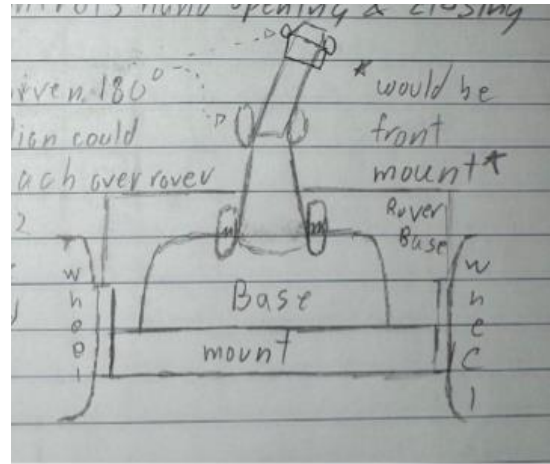


Figure 10: Front View Sketch of Arm Design

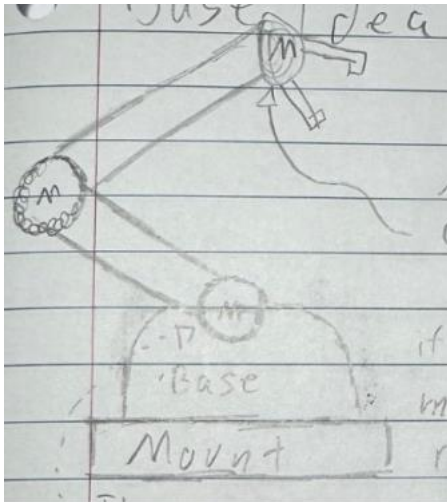


Figure 11: Side View Sketch of Arm Design

The initial design utilizes a multi-jointed arm driven primarily by Brushless DC Motors (BLDCMs). Each major joint is responsible for a single degree of freedom, allowing the arm to extend, raise, lower, and position the end effector. The concept also identifies the possible need for multiple BLDC motors at the shoulder joint depending on the final payload capacity and torque requirements determined during future testing. Since the shoulder joint experiences the greatest bending moment while lifting objects away from the rover, additional actuation or gearing may be required to safely support the arm throughout its operating range.

Mounting System:

One of the primary design considerations identified during early brainstorming was the mounting location of the robotic arm. The concept proposes a dedicated mounting platform attached directly to the existing rover chassis rather than replacing major structural components. This approach minimizes modifications to the inherited rover while allowing future teams to remove or redesign the arm without affecting the remainder of the platform.

The mounting system will likely house the primary rotational gearbox as well as the motor responsible for rotating the arm about its vertical axis. Locating the gearbox within the mounting assembly

reduces the amount of moving mass located farther from the rover and allows structural loads to be transferred directly into the chassis. The team also identified that the current rover mounting platform may require extension in length to properly support the arm and distribute loading over a larger area. Determining the minimum amount of modification necessary while maintaining sufficient rigidity will be one of the primary engineering challenges during prototype development.

The Software Design

HTTP Requests

Architectural Overview and Communication Topology

The software control framework of the RE-RASSOR platform employs a decoupled, distributed node communication topology to provide safe isolation of high-level remote user interface inputs from low level electromechanical execution. This network architecture employs an explicit HTTP REST API and asynchronous execution threads to avoid blocking conditions on the main telemetry loops.

It begins at the user interface (the React native or web application hosted on a client machine on the local net). The application communicates with the rover or its attachments through structured HTTP POST requests over a local network (via a private router) to the rover's main processing board (Raspberry Pi or Le Potato) running on a specific IP address on port 5000.

The High-Level Network Bridge: Flask and SocketIO Core

Server.py is the entry point of the microprocessor. It instantiates a multi-threaded web application using the Flask framework augmented with Flask SocketIO. This node is the main gateway from the web to ROS2.

The REST API Entry Point: The root URL path (POST /) functions as a unified command dispatcher. It processes multi faceted incoming payloads containing actions for wheels, shoulders, arms, drums, or autonomous targets.

Payload Verification and Normalization: To enforce system integrity and safeguard physical components from invalid operational states, the server feeds incoming JSON requests immediately into a rigid schema validator. The verification system strictly parses fields using predefined structural constants such as `WHEEL_ACTION_KEY = 'wheel_action'`, `SHOULDER_ACTION_KEY = 'shoulder_action'`, `FRONT_ARM_ACTION_KEY = 'front_arm_action'`, and `BACK_ARM_ACTION_KEY = 'back_arm_action'`. If an explicit action

violates defined types, such as passing arbitrary float ranges instead of the strict discrete bounds enforced by ArmAction (LOWER = -1.0, STOP = 0.0, RAISE = 1.0). the server aborts execution, throwing a VerificationError and responding with an HTTP 400 Bad Request status code. Validated inputs are mapped directly into structured runtime objects via command.py.

ROS2 Workspace Integration and Publisher Instantiation

Once a payload is verified, server.py converts the web variables into native ROS2 messages. The server spins up a dedicated ROS2 node lifecycle thread that owns explicit publishers for core motion topics:

- **wheel_publisher:** publishing geometry_msgs/msg/Twist to /ezrassor/wheel_instructions
- **shoulder_publisher:** publishing to /ezrassor/shoulder_instructions
- **front_arm_publisher / back_arm_publisher:** publishing to /ezrassor/front_arm_instructions and /ezrassor/back_arm_instructions

By executing the flask thread dynamically as a background daemon. The server guarantees that incoming consumer HTTP requests are captured asynchronously without introducing jitter or latency to the low level ROS2 compilation executor.

The Low-Level Serial Translation Layer

The server.py reads network traffic and converts it to ROS2 topics, but the conversion to hardware voltages is done by a separate downstream bridge script: serial_driver.py. This node is a high frequency subscriber that subscribes specifically to the server's mechanical Twist topics.

Physical Bus Addressing: The driver establishes serial links over USB buses mapped using persistent symbolic links created in the Linux subsystem (/dev/arduino_wheel and dev/arduino_drum) operating at a standardized transfer rate of 115200 baud.

Binary Serialization and Wire Protocol: Microcontrollers lack the computational efficiency to fluidly ingest full high level string payloads or floating-point structures under real time constraints. Therefore, serial_driver.py acts as a data compression encoder, flattening continuous Twist parameters down to single discrete, 8-bit hexadecimal opcode bytes before sending them over the wire. The established binary communication wire protocol maps as follows:

Target Device Bus	Monitored Input Parameter	Computed Hex Opcode	Intended Outcome
/dev/arduino_wheel	linear.x > 0.0	0x01	Forward Drive
/dev/arduino_wheel	linear.x < 0.0	0x02	Reverse Drive
/dev/arduino_wheel	angular.z > 0.0	0x03	Left Spin
/dev/arduino_wheel	angular.z < 0.0	0x04	Right Spin
/dev/arduino_wheel	linear.x == 0 & angular.z == 0	0x00	Full Braking
/dev/arduino_drum	msg.linear.y > 0.0 (Shoulder)	0x05	Raise Front
/dev/arduino_drum	msg.linear.y < 0.0 (Shoulder)	0x06	Lower Front
/dev/arduino_drum	msg.angular.y > 0.0 (Shoulder)	0x07	Raise Rear
/dev/arduino_drum	msg.angular.y < 0.0 (Shoulder)	0x08	Lower Rear

Table 1: [1][5] Existing Binary Wire Protocol Mappings

Transition Strategy: Refactoring the HTTP Schema for the Scaled Robotic Arm

While the inherited HTTP bridge structure functions reliably for the legacy system, it is custom tailored exclusively to support dual drum excavation systems (front_arm_action, back_arm_action, front_drum_action, back_drum_action). Because our current senior design group is pivoting the rover's utility to manipulate solid 3D printed pavers using a single, multi-axis scaled articulating arm, our team must execute an aggressive engineering refactor of this API ecosystem.

Expansion from Binary Opcodes to Direct Kinematic Coordinates:

Our new arm relies on lightweight Brushless DC Motors matched with precise magnetic field encoders. A simple binary 'RAISE' or 'LOWER' opcode (0x05 / 0x06) cannot safely coordinate multi joint interpolation or precise endpoint placement. The REST endpoint must be extended to accept absolute geometric position states or joint angles.

Refactoring the Middleware Chain:

- **1. request.py Validation Modifications:** We will strip out the binary constraints checking for front/back drums and append an array validator checking for the existence of explicit floating-point values for each targeted arm joint angle.
- **2. command.py Translation Enhancements:** The ArmAction class will expand from a basic discrete enum into a comprehensive kinematic state class capable of holding floating point structures.

Integration of Previous Requests

Analysis of Inherited Repository Branches

One of the group's main architectural challenges is reconciling overlapping and competing codebase frameworks developed by previous teams. Looking at the inherited workspace, the code is organized into two distinct operational pipelines: a localized Python web to serial bridge and a native C++ ROS2 framework leveraging low level serialization or outbound HTTP workers.

The Python Web to Serial Ecosystem: This pipeline utilizes server.py to instantiate an inward facing Flask REST and SocketIO web server running on port 5000. It handles remote string-based request configurations, parses them through an explicit validation engine (request.py), and converts them into standardized ROS2 topics. Downstream, a secondary Python node (serial_driver.py) subscribes to these topics and uses the pyserial library to write discrete 8-bit binary hex opcodes to two physical USB mount paths: /dev/arduino_wheel and /dev/arduino_drum.

The Native C++ Client/Driver Workspace: In stark contrast, the second branch has very structured C++ nodes. The motor_controller.cpp node is not a server. It is an HTTP client. It creates a local watchdog and fires off an asynchronous libcurl worker thread in the background (httpWorker) to push outbound JSON packets away from the ROS node to a hard coded remote target IP.

Conflict Resolution and the Unified Integration Strategy

To scale our robotic arm without breaking system stability, our team needs to have a definitive integration baseline. Keeping the Python Flask server loop and the C++ outbound curl injector with serious network loopback hazards, variable racing, and unnecessary command

latency. We have decided to standardize high level communications infrastructure around the Python Server Environment (server.py), while migrating low level motor coordination to a specialized serial package inspired by the C++ Driver Framework (serial_motor_controller.cpp).

Phases of Integration:

Phase 1: Deprecation of Outbound C++ HTTP Hooks

Because our control interface will be embedded directly within the preexisting React Native mobile/web app, the rover does not need to send outbound HTTP requests to itself via libcurl. The motor_controller.cpp outbound loop will be deprecated. Instead, the React Native front end will post commands directly to the server.py REST API entry point.

Phase 2: Preserving the 2023 ROS2 Workspace Core

We will preserve the native server.py background daemon architecture. This ensures that real world coordinate tracking and telemetry streams can still be pushed back to the client UI at high frequency using the integrated SocketIO WebSocket protocols (subscribe_odom, subscribe_map) without starving processing threads on the onboard microprocessor.

Phase 3: Hardware Bus Separation and Port ReMapping

The legacy C++ and Python serial architecture assume the presence of two low level controllers: a wheel controller and an excavation drum controller. Our multi-joint robotic arm requires an entirely independent control space. Linux rules will be appended to the microprocessor to register a new symbolic port: /dev/arduino_arm.

Scaling the UI and Web Styling Format

To ensure a clean user experience, our software modifications adhere to the styling and UI components already established in the inherited React Native application.

Component Reusability:

The legacy UI implements joystick interfaces and directional button grids wrapped in styling classes that handle responsive touch scaling and network timeout safety. We will clone these styling blocks to create a dedicated 'Arm Manipulation' control dashboard within the application workspace.

Integration of Previous UI Design

Our design for the updated User Interface (UI) is going to be based heavily on the existing UI from previous teams that worked on this project. The previous UI does not have the required features to fully and effectively control the robotic arm that we are planning to integrate with the rover. However, it provides a basis on which to build those features. Instead of designing a new interface from the ground up, we intend to extend the existing software to incorporate the features needed for the arm, while maintaining the usability of the current features.

This interface uses HTTP requests containing JSON formatted objects (as described above) to communicate between the rover's microprocessor and the laptop or other computer running the control interface. Existing controls include directional movement with tank steering, speed adjustments, and a stop button. The control interface also includes features for status monitoring, as well as access to the camera video feed. The below guides show how these controls are arranged in the existing rover driving and camera screens of the UI.

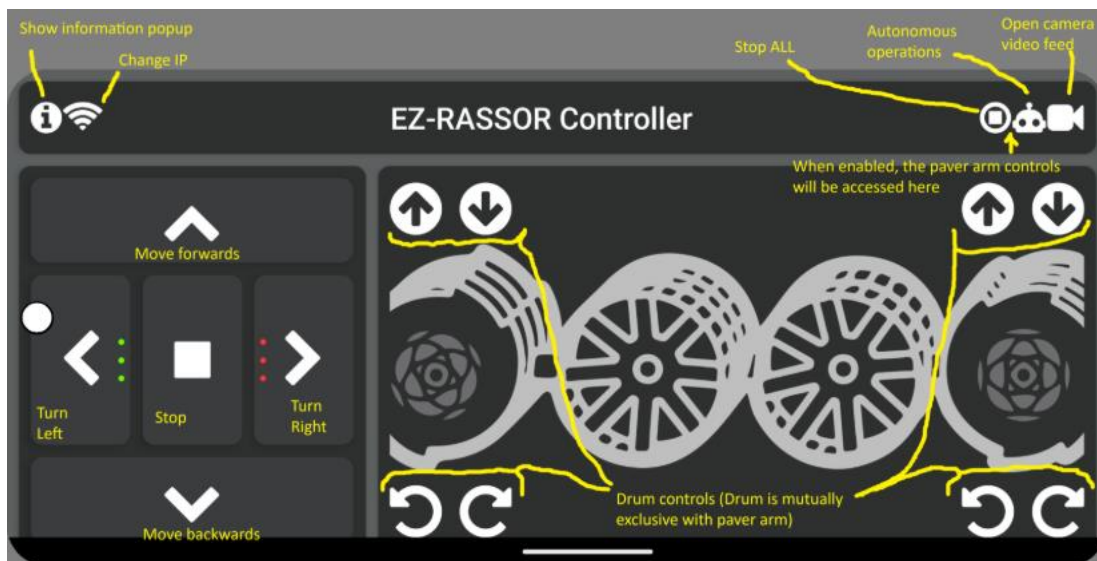


Figure 12: [5] Guide to Elements of the Existing Driving UI Screen

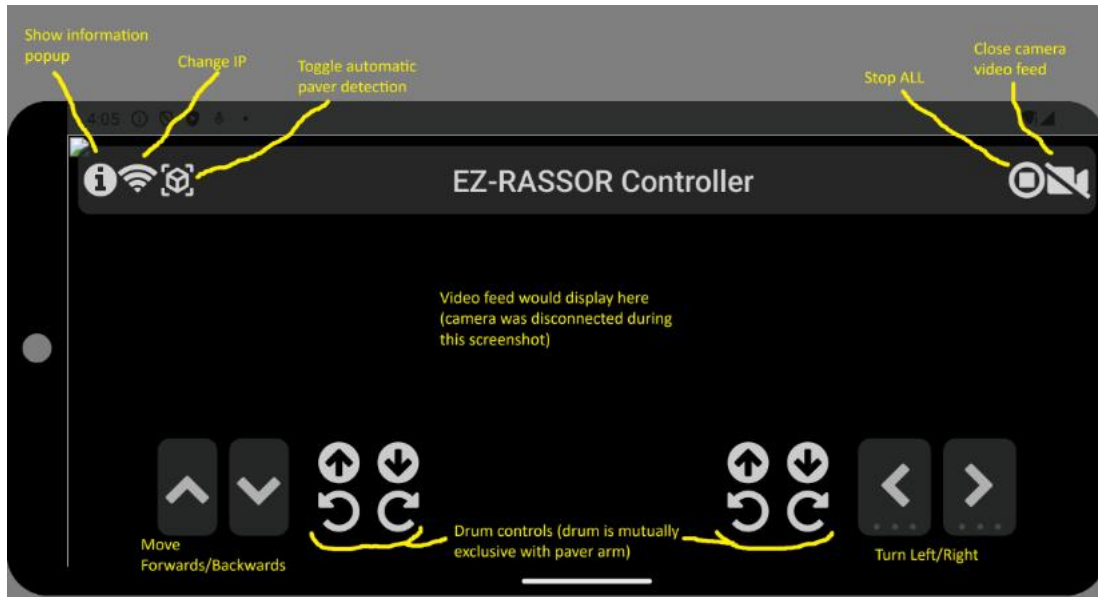


Figure 13: [5] Guide to Elements of the Existing Camera UI Screen

As the existing interface and its features have already been verified to function in extensive testing by previous teams, introducing significant changes beyond the required additions has been deemed an unnecessary effort with minimal benefits.

The existing control interface does include a component for the control of a robotic arm system. The code to access this component has simply been commented out of the overall interface. However, this component has multiple issues and will need to be adjusted to fix those issues. Most of these issues stem from the component being developed for a different arm design than the one we are planning to use.

Most notably, the component is developed to work with an arm design with a different arrangement of controllable joints than our design. This is especially important, as the arrangement of controllable joints is extremely important to effectively communicate to the user about which actions will have which results.

To be more specific, while three of the arm joints are approximately the same between our arm design and the previous arm design, the fourth is different between the two. The previous arm design had a controllable wrist, and a static end effector. However, our design has a static wrist and a controllable end effector.

As another issue, the graphical elements for the existing component do not represent our design for the robotic arm and will likely be confusing if not replaced with assets that accurately reflect the

current arm design. The graphical elements used in the component also do not match the rest of the UI stylistically, being untextured renders of 3D models.



Figure 14: [5] Example of Render Used in Previous Arm UI

Additionally, the existing component will require some further polishing, as it currently lacks any additional features beyond the bare minimum controls. Though, this will be relegated to a stretch goal, as these other features are not strictly necessary for functionality.

Another major consideration relating to the integration with the existing UI is trying to match the new UI elements stylistically with the existing ones. The existing interface utilizes a clean black, gray, and white color palette with clearly separated control panels and straightforward navigation. Most of the icons representing the rover or its parts use simple 2D symbols instead of 3D models. Using the same visual design language will make it easier for users familiar with the existing interface to use the extended version we develop.



Figure 15: [5] Example of the 2D Symbols We Need to Replicate for the Arm

Alternatives & Considerations in the Initial Phase

Mounting Area

There have been multiple alternatives considered for many of the design features described in this document. These alternatives were rejected for various reasons in favor of the chosen design features, which will be discussed here.

There were four alternative ideas considered for the location where the arm would be mounted on the rover:

- Top mounting
- Back mounting
- Double mounting
- Bottom mounting

Top mounting of the arm was rejected for the reason that it would likely interfere with usage of the power switch and charging port. Both of these components are located at the top of the rover and would likely need to be moved if the arm was placed there.

Back mounting the arm was rejected since it would require a longer arm to reach the front of the rover. It would also slightly interfere with the fan, as that component is located on the back of the rover.

Double mounting consists of arms mounted on both the front and back of the rover. This method was rejected because it would require us to use twice the number of materials. It would also still interfere with the fan component at the back of the rover. The second arm would also require additional controls on the User Interface, draw additional power from the battery pack, take additional network and processor bandwidth to send commands, and increase the rover's weight. The only benefit of this alternative would be the removal of any requirement for a counterweight, as the arms would balance each other's weights. If this route is taken in our design later, it will be due to issues with said counterweight; however, this was considered not ideal for the initial design phase before we had a completed prototype.

Bottom mounting the arm was trivially rejected for the reason that it would reduce the rover's already low clearance and significantly impair movement. It would also require significantly more effort and engineering to create a design that could successfully pick up the pavers.

Materials

For this, the only two alternatives were worth considering over the chosen PETG plastic: Polylactic Acid (PLA) and Polyvinyl Chloride (PVC) plastic.

PVC is more rigid and difficult to warp in comparison to PETG and is commonly used in classic robotic applications like those found in competitions like FRC. However, PVC requires extensive safety features as the fumes it produces during the printing process are hazardous. It can also be difficult to obtain a desktop 3D printer like our model and can result in degradation of the printer's nozzle over long

term use, making it impractical. This would reduce us to using prefabricated tubing and mechanical components not in the scope of this project

PLA is safer than PVC and lighter than both PETG and PVC; however, it was rejected for similar reasons as it also has a low melting point and experiences severe warping and melting under relatively low heat. This was showcased in several of our predecessor's designs both in operation and environmental circumstances. Multiple rovers and components were documented to melt under Florida heat or when motors are put under strain. This makes components made from it less reliable and more fragile, resulting in it being more costly in the long term. However, its affordability and relative safety make it ideal for test prints. As a result, all initial trials on new components were conducted with PLA to ensure that sizing and proportion of components was accurate before utilizing the more costly final material

	Cost Per 1KG Spool (USD)	Warping Temperature in Celsius	Toxicity	Weight per Cubic CM
PETG	18 - 30	80	No	1.27 g/cm ³
PLA	15 - 25	55 - 60	Minimal	1.24 g/cm ³
PVC	N/A	80 - 87	Yes	1.35–1.45 g/cm ³

Table 2: Relevant Material Properties of Three Plastic Types

Mechanical Components

Motors

Our design utilizes mainly BLDC motors. These motors are light weight, constructed with 3D printed components and a small gimble motor which results in an overall motor weight of 80g. Our main alternative was the motor used by our predecessors: the stepper motor. Stepper motors are constructed primarily with metal components, making it roughly three times the weight of the compact and efficient BLDC motor. While this makes it sturdier and more capable of dealing with more pressure; it also makes the overall arm far too heavy and thus results in the weight issues seen in team 2022's design.

We elected to use it in a singular place to reduce the overall weight of the arm: the base gearbox. This gearbox and its motor needed to be capable of functioning under the weight of the entire arm mechanism. This scenario makes the stepper the ideal candidate given the information we had in this design phase as it balances the need for overall stability with the looming constraint of weight.

The Base Cyclical Gearbox Approach

Multiple alternatives to the cycloidal gearbox were considered for use in the shoulder joint of the arm. These being: the worm gear, bevel gears, and screw gears. These alternatives were considered during the initial designing phase as a way to fix the issue of top-mounting the arm interfering with the power switch and charging port, as they would allow the motor to be placed away from those components. However, the worm gear and screw gears were deemed to be unnecessarily complicated, and the bevel gears would require compromising the torque ratio of the joint, so these options were discarded along with the top-mounting idea. Instead, the current design works much like the gearboxes of the wheels. We have taken those components and resized and flattened them to make the system for the base's rotation.

Forward Compatibility: Preparing for Computer Vision & Stretch Goals

Aside from the project's primary goal. If we got the chance, we also have some stretch goal in mind: Autonomous Object Pickup. Prior teams attempted computer vision using external hardware like the NVIDIA Jetson Nano but struggled heavily with physical integration and documentation.

To bridge this gap for future iterations, we could intentionally implement `server.py` that leaves native WebSocket channels open for streaming live camera depth matrices (`subscribe_points` and `subscribe_depth_image`). By broadcasting the downsampled XYZ camera coordinates as base64 encoded grayscale arrays directly through the web port, future teams can inject image tracking models (such as ArUco marker detection scripts) straight into this server loop. When a paver is detected, the vision script can calculate the target coordinate offsets and execute automated movements by passing coordinate payloads back through our newly refactored JSON entry points, bypassing the need for manual driver intervention.

Management Considerations

When considering timeline management, our team, sponsor, and professor agreed that our team should utilize a Kanban style for this project. While this is not typical of Computer Science for senior design projects undertaken at the University of Central Florida, it was deemed better than its alternative Scrum for a few reasons. Scrum is a more linear style of agile development. To work, it requires a full understanding of all tasks at the beginning of the project in order to be organized into sprints. These sprints are then taken up by the team in a set timeline to ensure project completion. While it is a sound development process, it was determined that for an experimental and research-based project like RE-RASSOR, it would be less than sufficient to manage our project as our plan relies upon multiple factors working correctly to be able to commence with the next tasks. This is why Kanban was chosen. Instead of sprints, our method focuses on classifying tasks by stages of completion and doing frequent analyses of schedule to ensure that foundations are set before continuing. By using this process, we are able to add in

new tasks easily when a design fails or error occurs as will likely happen due to the nature of our project and the experience of development teams before us.

Initial Feasibility Analysis

Based on the team's research, sponsor guidance, and analysis of previous RE-RASSOR development efforts, we believe the baseline deliverables are feasible within the scope of this project. Existing rover hardware, prior documentation, and lessons learned from earlier arm prototypes provide a strong technical foundation that reduces development risk compared to starting from a blank design. While several previous robotic arm implementations have not progressed to long-term operational success, those efforts have produced valuable research that informs our design decisions and helps identify potential challenges before fabrication and testing begin.

The team also recognizes several risks that may impact development. Although members have foundational programming experience, some are still developing proficiency with skills such as computer-aided design (CAD), 3D printing workflows, and the HTTP-based communication used by the rover's control system. These learning curves introduce additional uncertainty into the development timeline. To mitigate these risks, the project will emphasize iterative prototyping, structured testing, and comprehensive documentation of design decisions, test results, and failures. As this is fundamentally a research project, unsuccessful design iterations are expected to provide valuable engineering data. Thorough documentation will ensure that both successful solutions and identified limitations can be leveraged by future RE-RASSOR teams, contributing to the long-term advancement of the platform regardless of the final outcome of this individual project.

Initial Considerations of Stretch Goals & Feasibility

The primary objective of the project is to build a rover mounted arm that can lift and manipulate 3D printed pavers through the existing control system. Stretch goals should be considered after the baseline arm is functioning and approved by the sponsor. Each stretch goal should be judged by the amount of redesign it would require, the risk it creates for the required deliverable, and the amount of value it adds to the RE-RASSOR platform. A stretch goal is more realistic if it builds upon hardware or software that already exists in the baseline design. A stretch goal is less realistic if it depends on several unfinished systems working at the same time.

Autonomous Object Pickup

Autonomous object pickup would allow the rover to identify a paver and pick it up without direct manual control. The rover would need to detect the object, estimate its position relative to the arm, move

into a usable position, and run the pickup sequence. This would make the rover more interesting for educational demonstrations by connecting sensing, control, and physical manipulation.

This stretch goal has low feasibility for the current project scope. It depends on several systems being reliable before autonomy can be attempted. The camera would need a clear view of the paver from a useful angle. The software would need to detect the paver under changing lighting conditions. The rover would need to position itself close enough for the arm to reach the target. The arm would also need repeatable movement, so the pickup sequence produces the same result each time. If any of those pieces are inconsistent, autonomous behavior will be difficult to trust.

The largest issue is that autonomous pickup depends on mechanical repeatability. A human operator can correct small alignment problems during manual control. An autonomous routine would have to rely on the arm moving in a predictable way. If there is play in the shoulder, flex in the mount, or inconsistent gripper contact, the software may correctly detect the paver and still fail during pickup. For that reason, autonomous pickup should not be attempted until the arm can already perform the paver task reliably under manual control.

This goal could still be supported through design choices made during the baseline project. We can leave room for a future camera mount, document the arm's range of motion, and organize the software so future commands can be added cleanly. If the current team has extra time, a limited version of the goal could be explored through basic image viewing or simple object detection tests. Full autonomous pickup should remain in long term work for a future RE-RASSOR team.

Additional Wrist Joint for Rotation

An additional wrist joint would allow the gripper to rotate along another axis. This could help the arm approach objects from more angles and may support future end effectors. A fixed gripper may work for pavers if the rover approaches them consistently. More complex objects may require the gripper to rotate before it can make contact correctly.

This stretch goal has medium feasibility. It extends the current arm concept, but it affects the most sensitive part of the design. Any added weight near the wrist increases the load carried by the shoulder and elbow. The joint would also require another motor connection, more wiring through the arm, new control behavior, and more CAD work. These changes are manageable if the baseline arm has enough weight margin. They become risky if the shoulder joint is already near its practical limit.

The main benefit of this goal is improved end effector control. A wrist rotation joint could make the gripper more useful when the rover does not approach the paver from the ideal direction. It could also

make future attachments more practical, especially tools that need to face a certain direction before contacting the ground or an object. For example, a scoop or collection tool may need a different orientation than a simple paver gripper.

The decision to add a wrist joint should come from baseline testing. If the gripper can pick up and place pavers consistently without extra rotation, the wrist joint should remain a future improvement. If testing shows that the gripper struggles mainly due to an approach angle, then wrist rotation becomes more useful. This goal should only be pursued after the shoulder, and elbow joints show that they can handle the added load.

Interchangeable End Effector Attachments

Interchangeable end effectors would allow the arm to use different tools without redesigning the full mechanism. The baseline design will focus on a gripper for 3D printed pavers. A modular connection would let future teams create alternate grippers, scoops, collection tools, or classroom demonstration attachments.

This stretch goal has high feasibility since it builds directly on the required gripper work. The arm already needs an end effector, so the connection point can be designed with future replacement in mind. The main challenge is making the connection strong enough that it does not loosen during lifting. The attachment system should not add unnecessary weight to the wrist or weaken the baseline gripper. A simple and rigid connection would be more valuable than a complicated system that is difficult to print or assemble.

This goal also fits the long-term purpose of RE-RASSOR. The rover is meant to be reused by future student teams, so a modular end effector interface would make future work easier. A later group could design a new attachment while keeping the same arm structure. That would reduce the number of redesigns required for future projects and make the arm more useful as an educational platform.

The simplest version of this goal would be a standardized mounting pattern at the end of the wrist. The paver gripper would use that pattern, and future tools could be designed around the same interface. The attachment does not need to be complex to be useful. A reliable screw based or keyed connection could give future teams enough structure to build from without adding unnecessary mechanisms to the current prototype.

This should be one of the first stretch goals considered after the baseline arm works. It adds value without requiring object detection, autonomy, or major changes to the rover software. It also encourages

better documentation, since the mounting interface would need clear dimensions and assembly instructions for future use.

Camera Integration into End Effector

A camera near the end effector would help the operator see the paver and gripper during pickup. The main rover camera may not show the contact point clearly when the arm is close to the ground. A closer view could make manual operation easier and reduce failed pickup attempts during demonstrations.

This stretch goal has high feasibility if the camera is first used as a manual viewing aid. Camera placement would be the main design issue. The camera should show the gripper's contact with the paver without blocking the gripper or sitting in a location where it could hit the ground. Wiring would also need to move with the arm without catching on the joints. The camera mount should stay lightweight, so it does not add unnecessary load to the wrist.

The camera would also support testing. During early paver pickup attempts, a close view of the gripper could help us understand why a lift succeeds or fails. If the paver slips, the camera may show whether the issue came from poor alignment, weak contact, or an awkward approach angle. This would make the camera useful even before object detection is attempted.

The difficulty increases if the camera is expected to support object detection. A camera feed for a human operator is much simpler than a vision system that identifies pavers and supports autonomous pickup. Object detection would need software that can recognize the paver and produce information the rover can use. That work depends on camera placement, image quality, lighting, and repeatable arm behavior.

A practical approach would be to mount and test the camera for operator viewing first. Once that works, the same camera location could be evaluated for future object detection. This keeps the stretch goal realistic while still moving the project toward future autonomy. If time is limited, the manual camera view should be prioritized over computer vision.

Complex Counterweight System

A counterweight system could help address rover stability while the arm is extended. The preliminary design considers a rear storage bin that could also act as a counterweight. This concept becomes stronger if the added mass has a function, such as storing collected pavers or supporting future regolith style demonstrations.

This stretch goal has medium feasibility. A simple rear storage bin or passive counterweight could be designed without major software changes. A dumping or collection mechanism would require

more mechanical design and more testing. It could also add enough weight to affect driving performance. We should decide whether a counterweight is necessary based on stability testing rather than adding one by default.

The main value of the counterweight system is stability. If the arm causes the rover to tip or lift a wheel during paver handling, a rear counterweight may help keep the rover balanced. A storage bin could make that added weight more useful by giving it a secondary purpose. Instead of adding unused mass, the rear section could support collection or demonstration tasks.

The main risk is overcomplicating the rover. Extra weight may help balance the arm but it can also make the rover harder to drive. A more complex rear mechanism could introduce new failure points and take time away from the arm itself. A passive counterweight or fixed storage bin is more realistic for the current scope. A moving dump mechanism should be treated as future work unless the baseline arm is finished early.

The counterweight concept is a good topic for future teams if the current semester does not leave enough time. A storage or regolith collection system would connect well to the larger RE-RASSOR purpose, but it should not distract from completing the arm. The best outcome for our team may be documenting the concept clearly, recording stability test results, and leaving enough design information for future development.

User Interface Design Alterations for Precise Control

The new UI module could replace the existing directional buttons with a clean, slider-based joint coordinate array accompanied by a dedicated toggle switch for operating the end effector claw. Rather than repeatedly pressing directional buttons to gradually move each joint, every degree of freedom would be represented by an individual slider corresponding to a specific servo or motor within the robotic arm. This approach would allow operators to visualize every joint position simultaneously while making fine adjustments with significantly greater precision.

This approach would provide a more intuitive schema for controlling rotational joints throughout the arm and rover due to directly representing angular position rather than relative movement and positioning. As a result, this reduces the number of user inputs required to reach a desired position for a variety of complex tasks

Moving a slider would require dynamically recalculating the target angles in real time, grouping all updated joint positions into unified JSON string before dispatching them over the network to the Raspberry Pi or Potato. Sending the complete set of target coordinates rather than individual movement

commands reduces unnecessary network traffic while ensuring every joint receives synchronized position updates. This communication strategy would also simplify the software implementation on both the user interface and embedded controller by maintaining a consistent packet structure regardless of how many joints are being adjusted only after significant position changes or after the operator has completed slider movements, further reducing network congestion while maintaining a responsive control interface.

The use of coordinated positioning packets also creates opportunities for future software enhancements to the current source code. Rather than commanding individual motors independently, complete arm poses could be generated and transmitted as a single command. Not only would this simplify coordinated movements where multiple joints must move simultaneously to achieve a desired position or task but would also open up more possibilities or autonomy by utilizing sensors and cameras to direct these set poses enabling rover indecency with complex movements and tasks. This overall approach would improve motion smoothness, enable repetitive complex tasks, and create a foundation for advanced control algorithms such as trajectory generation or inverse kinematics in future versions of this project

This type of control scheme would have a variety of complications however in order to make it possible. Firstly, an option to recalibrate these target positions may be necessary as the rover's arm experience wear, maintenance, or hardware modifications throughout its operational life. Even small mechanical tolerance introduced to the robot's assembly and operation can produce slight derivations between commanded software position and physical orientation of each joint. A calibration feature integrated directly into the interface would allow operators to redefine home positions, adjust software offsets, and compensate for mechanical drift without modifying the underlying source code. Calibration information could be stored locally or within the rover's configuration files, allowing the system to maintain consistency positioning accuracy across multiple operating sessions.

Providing accessible calibration controls would also improve maintainability for future RE-RASSOR senoir design teams. Rather than requiring future developers to manually modify firmware parameters or hardcoded joint offset, a dedicated calibration interface would centralize these adjustments into a repeatable and documented process. This aligns with the project's log-term objective of creating software that is maintainable, extensive, and capable of being handed down to future development teams with minimal reconfiguration.

Additional sliders can be added to adjust the speed or sensitivity of various motors and joints. Different operational tasks often require different movement characteristics depending on the environment and desired level of precision. For example, rapid movement may be appropriate while

transporting the arm between general positions, whereas delicate manipulation tasks benefit from significantly slower and more precisely controlled joint movements. Allowing operators to dynamically adjust motor speed directly through the interface would eliminate the need to modify controller parameters during operation while providing substantially greater flexibility.

Sensitivity adjustments could further improve overall usability by controlling how aggressively slider movement translates into motor rotation. High sensitivity settings could allow experienced operators to move the arm rapidly across its full operating range, while low sensitivity settings would enable finer positioning during precise manipulation tasks such as collecting samples or aligning with smaller objects. These controls would make the interface more adaptable to operators with varying levels of experience while reducing the likelihood of accidental overshoot caused by aggressive control inputs.

Future interface revisions may also incorporate visual indicators displaying current joint positions, commanded target positions, communication status, and system health. Presenting this information directly within the operator interface would improve situational awareness while assisting with troubleshooting during testing. Color-coded indicators similar to that of a car's warning indication for features like tire, engine, and oil status could immediately notify operators of communication failures, invalid command values, or mechanical limits and rover health, reducing debugging time and minimizing the possibility of hardware damage resulting from unintended motion.

Another potential enhancement involves allowing operators to save and recall predefined arm configurations. Frequently used positions such as a home position, transport configuration, or others could be stored within the application and recalled using a single command. This capability would improve operational efficiency while ensuring consistent arm positioning throughout repeated testing procedures and demonstrations. Storing these configurations externally within configuration files would also simplify future expansion by allowing additional positions to be created without requiring software recompilation.

As future development continues, integrating a simplified graphical representation of the robotic arm into the interface may further improve operator confidence. As each slider is adjusted, a digital visualization could update in real time to reflect the expected joint configuration before movement commands are transmitted to the physical hardware. Such visualization would allow operators to verify intended motion, identify impossible joint configurations, and detect potential collisions before execution. This feature would become particularly valuable during remote operation where direct observation of the robotic arm may be limited.

Collectively, these proposed interface improvements would transform the current command-based control system into a comprehensive operator's workstation capable of supporting both routine operation and long-term maintenance. By emphasizing precision control, improved operator feedback, simplified calibration procedures, configurable movement characteristics, and enhanced visualization, the user interface would become significantly more intuitive while providing a scalable foundation for future software development. These enhancements align with the project's broader objective of producing a robotic system that is not only functional during the current development cycle but also maintainable, extensible, and capable of continued improvement by future Senior Design teams.

Stretch Goal Prioritization

Not every proposed enhancement provides the same value or carries the same level of implementation risk. Future development efforts should prioritize additions that maximize functionality while minimizing redesign of the baseline robotic arm. Features that build upon existing hardware, software, or documentation should generally be favored over those requiring entirely new subsystems.

The highest priority improvements are those that directly extend the baseline deliverable without significantly affecting the completed mechanical design. Examples include interchangeable end effectors, camera integration for operator viewing, and user interface improvements. These additions primarily leverage existing infrastructure while expanding the rover's capabilities and educational value.

Medium-priority goals include hardware modifications that require redesign of existing components but remain compatible with the established software architecture. Additional wrist articulation and passive counterweight systems fall within this category because they primarily affect the mechanical subsystem while providing measurable improvements to manipulator performance.

Lower-priority goals involve significant software and systems integration challenges. Autonomous object detection and fully autonomous manipulation require reliable performance from nearly every subsystem before implementation can begin. These goals should only be considered after extensive validation confirms repeatable arm motion, stable communication, and consistent mechanical performance.

Following this incremental development philosophy minimizes project risk while ensuring each enhancement builds upon a stable engineering foundation.

Suggested Long-Term Development Roadmap

While the primary objective of the current Senior Design team is to develop a functional rover-mounted robotic arm capable of manipulating 3D printed pavers, the broader vision of the RE-RASSOR

platform extends well beyond a single semester. As a continuing university research project, each team is expected to contribute meaningful improvements while documenting their work in sufficient detail for future teams to continue development. Rather than viewing the completion of this project as a final product, the current design should serve as a stable and well-documented engineering foundation upon which additional mechanical, electrical, and software capabilities can be incrementally developed.

The roadmap presented below outlines one possible sequence for future development. The order is based on engineering risk, subsystem dependencies, and the expected educational value of each enhancement. By completing lower-risk improvements before introducing increasingly complex functionality, future teams can minimize redesign while continuously expanding the capabilities of the RE-RASSOR platform.

Phase 1: Baseline System Validation

The first phase should focus on refining and validating the baseline robotic arm. Before additional functionality is pursued, the manipulator, communication system, software, and user interface should demonstrate consistent and repeatable performance through structured testing. Completing this phase provides the stable foundation required for all future development while producing comprehensive documentation for subsequent teams.

Phase 2: Modular Platform Enhancements

Once the baseline system has been validated, development should prioritize improvements that build directly upon the existing platform without requiring significant redesign. Potential enhancements include interchangeable end effectors, user interface improvements, camera integration for operator viewing, and additional diagnostic tools. These features provide meaningful capability increases while maintaining relatively low implementation risk.

Phase 3: Mechanical Capability Improvements

Future mechanical improvements should focus on expanding the arm's functionality through additional degrees of freedom, refined joint geometry, optimized counterweight systems, and improved structural components. These changes should be guided by data collected during baseline testing to ensure redesign efforts to address verified limitations rather than assumed deficiencies.

Phase 4: Intelligent Assistance

Following successful validation of the mechanical platform, software development may expand into intelligent operator assistance. Features such as object recognition, visual alignment aids, assisted positioning, and semi-autonomous task execution would improve usability while still maintaining operator oversight. These capabilities build naturally upon the stable hardware and software developed during previous phases.

Phase 5: Autonomous Manipulation

The final stage of long-term development would be fully autonomous for object manipulation. At this point, the rover would independently detect a target, calculate an approach, execute the pickup sequence, and verify successful completion. Because this objective depends upon reliable performance from every major subsystem, it should remain a long-term research goal rather than a primary deliverable for the current project.

For More Information on Our Considerations of Users During Our Design See
the Administration Section of this Document

Design Diagrams & Descriptions

Use Case Diagram

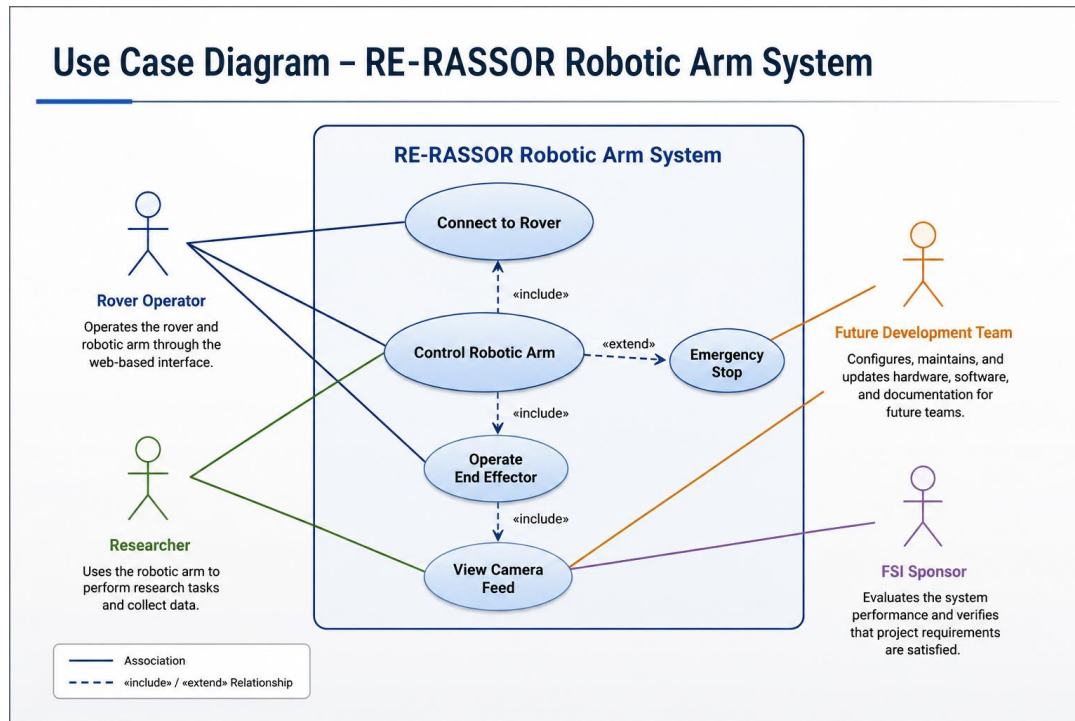


Figure 16: Use Case Diagram - RE-RASSOR Robotic Arm System

This figure illustrates the primary interactions between the RE-RASSOR robotic arm system and its stakeholders. The rover operator serves as the primary user, utilizing the web-based interface to connect to the rover, control the robotic arm, operate the end effector, and issue an emergency stop when necessary. Additional stakeholders, including researchers, future development teams, and the Florida Space Institute sponsor, interact with the system through maintenance, experimentation, and project evaluation activities. This diagram defines the system's functional requirements from the perspective of each stakeholder and establishes the foundation for the subsequent workflow and architectural diagrams.

Activity Diagram

Activity Diagram – Control Robotic Arm

Shows the workflow from the operator's request to the robotic arm's movement and feedback.

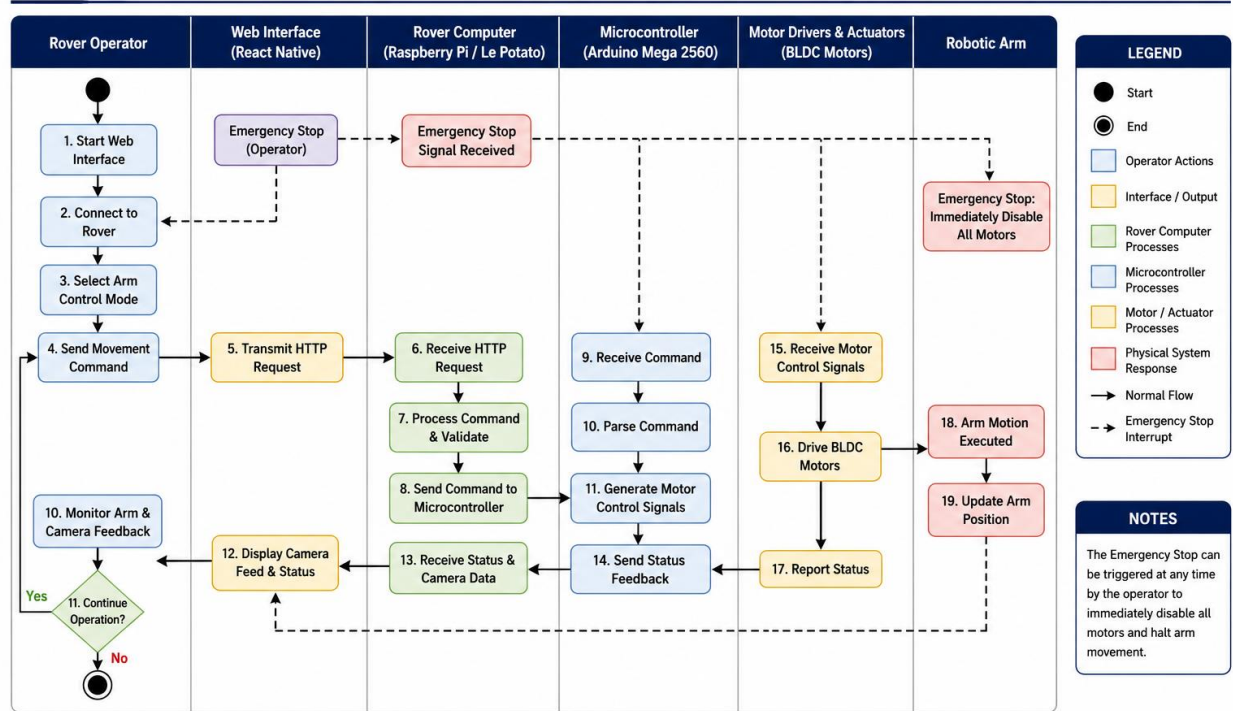


Figure 17: Activity Diagram - Control Robotic Arm

This figure illustrates the sequence of operations performed during normal robotic arm control. The workflow begins when the rover operator establishes a connection through the web interface and transmits movement commands to the onboard computer. Commands are processed by the Raspberry Pi (or Le Potato), forwarded to the Arduino Mega 2560, and translated into motor control signals for the BLDC motor drivers. Status information and camera feedback are continuously returned to the operator, creating a closed-loop control system. The activity diagram also identifies the emergency stop pathway, which allows motion to be halted immediately regardless of the current stage of execution.

Block Diagram

Block Diagram – RE-RASSOR Robotic Arm System

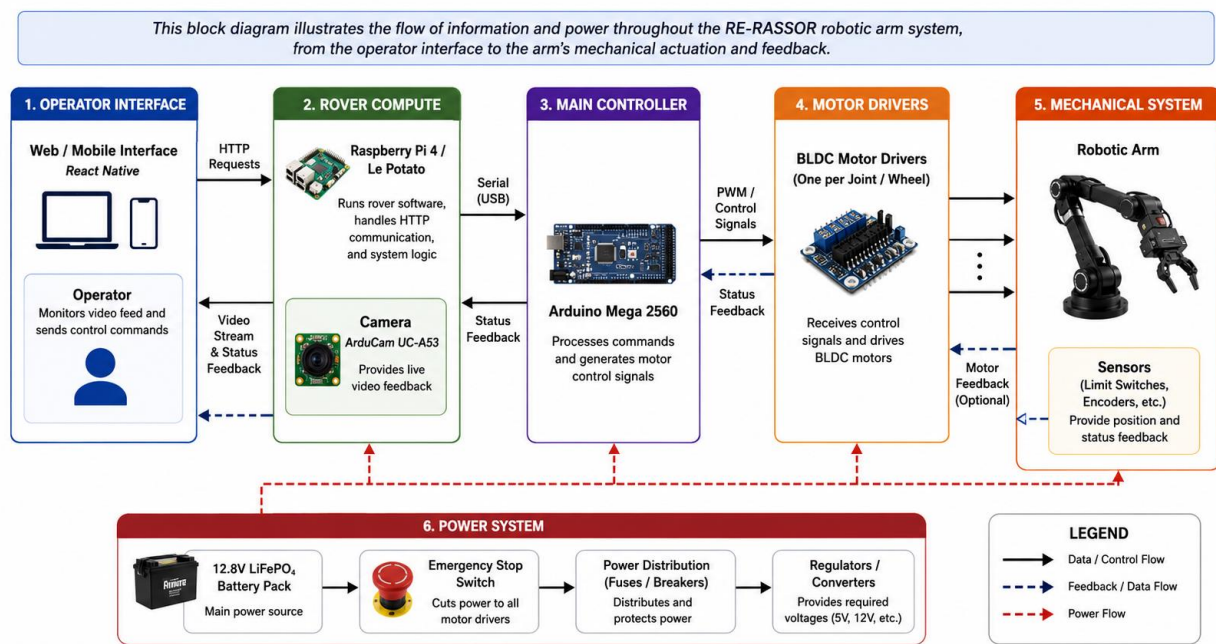


Figure 18: Block Diagram - RE-RASSOR Robotic Arm System

This figure presents the high-level architecture of the RE-RASSOR robotic arm system, illustrating the flow of control signals, feedback data, and electrical power between the system's major hardware components. Beginning with the operator interface, commands are transmitted to the rover's onboard computer before being processed by the microcontroller and distributed to the motor drivers responsible for actuating the robotic arm. Feedback from onboard sensors and the camera is returned to the operator to provide situational awareness during operation. The diagram also highlights the system's power distribution and emergency stop circuitry, demonstrating how hardware components interact to provide safe and reliable robotic arm operation.

Component Modifications Made During Development

The Mounting Plate

Version 1

Base Measurements: The overall size is 8 x 8 x 1 inches cube. The narrowest section located at the bottom of the platform measures at 6 x 8 x 2/3 inches cubed, and the widest section located at the top including the mounting wings are 8 x 8 x 1/3 inches cubes. The wings are an inch wide each with 3 M5 screw holes on the back where the platform meets the body of the rover

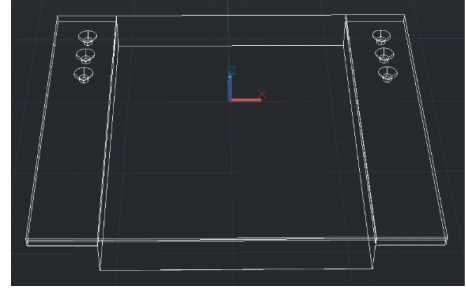


Figure 19: Version 1 Mounting Platform CAD Model

Failure Point: This version was a mockup constructed to ensure the accuracy of the measurements as existing information was not easily accessible, and manual measuring complex components tend to be inaccurate. While the top half of the platform fit the 8 inches width and 1-inch-tall clearance space granted by the rover's current design, the bottom was 20mm too wide to fit between the shoulder mounting sections. Additionally, the screw holes were too large for the existing rover's shoulder and thus had to be reduced

Version 2

Improvements on V1: All catastrophic sizing issues have been fixed. Additionally, this version was printed alongside the arm base. 4 M5 Screw holes were added to the original design to join the 2 components



Figure 20: Implemented Version 2 Mounting Platform

Possible Future Improvements: Currently the 6 M3 screws require M4 washers to fit snugly to the rover. Future efforts could be made to reduce the hole size, but the design currently can hold half the weight of the rover and thus should be secure enough for the Version 1 of the arm itself

Nonprinted Hardware:

- 6 M3 3.5mm Flathead Screws
- 6 M3 Bolts
- 6 M4 Washers
- 4 M5 (put finalized size here) Flathead Screws
- 4 M5 Bolts

The Arm Base

Version 1

Base Measurements & Purpose:

The first iteration of the arm base utilized 4 M5 holes to mount to the chassis of the RE-RASSOR rover, while 4 M3 35mm holes were used to mount to the actual stepper motor gearbox. The overall diameter of the large base circle measured 120mm, while the diameter of the raised cylinder is 90mm, with a thickness of 16mm. This means the inner hollow cylinder has a diameter of 74mm. The base circle has a 6mm fillet, along with 8mm tall to give support. Additionally, a small hole is extruded out to allow for the wires from the NEMA stepper motor to go out, and into the RE-RASSOR chassis. The hole is dimensioned from the official documentation on the NEMA 17 stepper motor.

The first version of the Arm Base had 4 raised pillars each with the M3 holes specified earlier. This was for supporting the original version of the gearbox created for the stepper motors. The purpose of this component is to support the base stepper motor and attach the stepper gearbox & arm base pivot. Additionally, there is a hole in the back to allow the stepper motor cables to easily go through it, into the rest of the RE-RASSOR chassis.

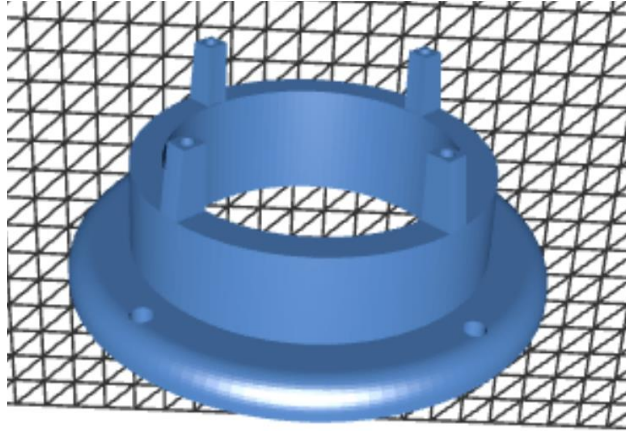


Figure 21: Version 1 of the Arm base.

Failure Point:

The main issue with version 1 is the 4 stilts that rise up. The original gearbox design had four diagonal mounting holes that supported this design. However, upon inspection in the FSI Laboratory, the 2022 BLDC team used a stepper gearbox with a much better design, allowing us to have a flat mounting surface instead of the 4 stilts.

Additionally, the first version was too low to fully support the specified motor we were using. This led to the stepper motor being exposed from the outside, meaning it would be unable to fit the gearbox. Additionally, if the motor is too low to the ground, the heat from it being used could cause the PLA test material to warp and deform.

Version 2

Improvements on Version 1:

The new version of the Arm base allowed us to simply raise a full hollow cylinder, instead of the 4 stilts. This makes the base much more stable and easier to mount the stepper gearbox. Additionally, the cylinder was raised to a total of 50mm from the ground. This gives it enough leeway to allow the stepper motor to mount properly, stay hidden from the outside, and have enough free space to not heat up the mounting plate discussed in the earlier section.

2026 RE-RASSOR Final Design Document

An additional feature added is 3D modeling a version of the stepper motor we are going to be using, based off of the NEMA 17 design specification. This was very helpful for the second version, as not only could we put this into an assembly with the arm base and the gearbox, we can also see a preview of what the final prototype will look like. This was how the version 2 of the arm base was dimensioned properly with the NEMA 17 stepper motor gearbox

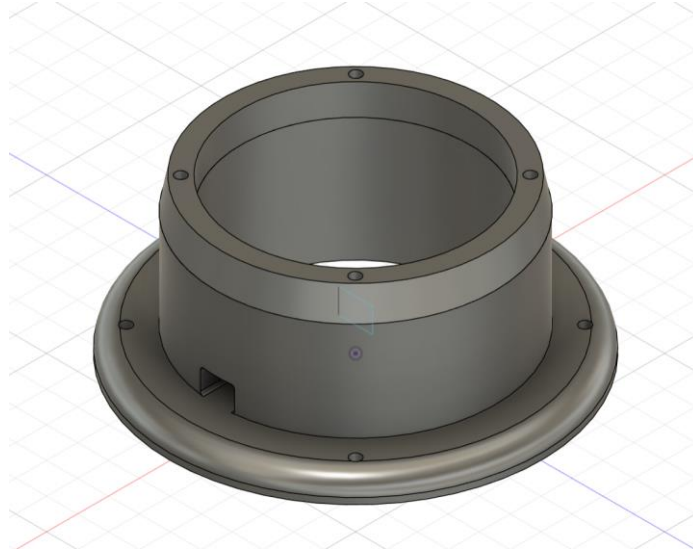


Figure 22: Version 2 of the Arm base.

Possible Improvements:

Since we haven't tested a fully prototyped version yet, the thickness of the bottom of the base could be increased to allow for more stability. Additionally, the thickness of the extruded cylinder walls may be too weak to connect to the gearbox with the full torque of the arm. If this is the case, we can increase the thickness to allow for more support and keep the M3 mounting holes in the same location to still allow the gearbox to connect.

Additionally, depending on the heat that the motor generates during testing and production, we may need to raise the walls of the base component, to allow the gearbox to mount higher. This would allow the motor to sit higher up, so the chance of the 3D print material deforming due to heat will be lower.

Total Non-Printed Hardware:

- 8x M3 3.5mm bolts.

- 8x M3 Washers.
- 1x NEMA 17 Stepper Motor.

The Arm Base Pivot Connector

Version 1

Base Measurements & Purpose:

The purpose of this part is to act as the connecting piece that connects the gearbox from the stepper motor to the actual arm mechanism. It is made to mount on the output platform of the gearbox that is meant for the base, which means it will spin around as the stepper motor spins.

The base of the pivot measures 55mm x 55mm x 5mm. This has 4x M3 3.5mm holes to connect to the output channel. On the sides of the pivot are mounting walls for the driving BLDC motor gearboxes. These have 4x 2.5mm mounting holes for the gearboxes, and a large 25mm hole for the shaft and rest of the motor.

In the center of the pivot is an extruded hole with a 5.5mm diameter to allow the BLDC motor shaft to pass through.

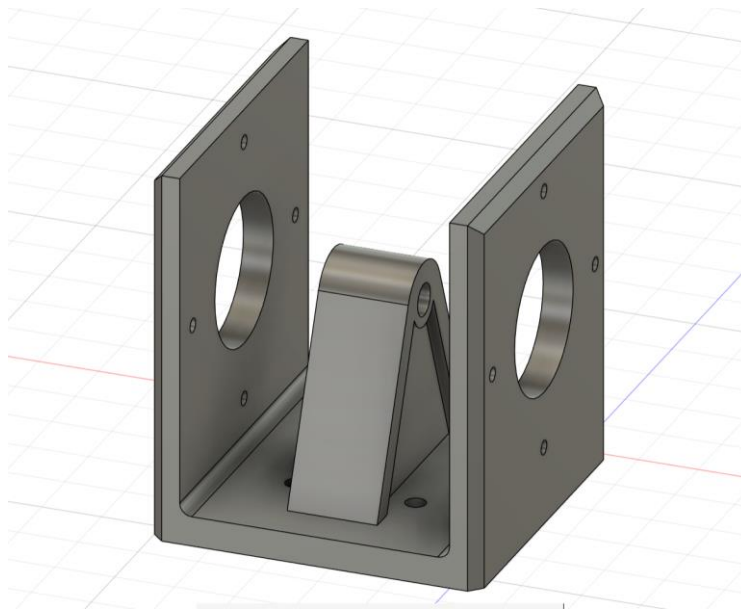


Figure 23: CAD Model of Arm Base Pivot Version 1

Possible Improvements:

The first major point of improvement would be the side support walls. These are meant for the BLDC motor to mount to; however, if it proves to be too heavy, we will have to improve it by adding more side support. Additionally, depending on how the cables lie from the BLDC motors, we may need some guide to prevent them from being tangled as the pivot spins.

Another point of improvement would be the center raised pivot. This part doesn't currently provide much structure for the rest of the arm that will stem from it. If the arm ends up being very heavy, we will have to extend the support from the base of the object.

The final point of contention is the BLDC motor gearbox itself. While we do have the 2025 version, if the gearbox itself needs to be changed for any reason, dimensions are altered. Parts such as this one would also need to be changed to match any changes made to the gearbox.

Total Non-Printed Hardware:

- 12 x M3 3.5mm bolts.
- 12 x M3 Washers.
- 1 x NEMA 17 Stepper Motor.
- 2 x BLDC Motors + Gearboxes

Planned Modifications to the User Interface

A total architectural and visual refactoring of the current RE-RASSOR user interface (UI) is required to accommodate the multi axis 3D printed robotic arm. The inherited UI from the previous senior design teams was custom built for dual counter rotating drum excavators and tank-steering drive controls. It lacks multi-joint position tracking, joint-space control sliders, inverse kinematic coordinate inputs, and real-time visual telemetry feedback necessary for the safe and intuitive operation of the manipulator.

Limitations of the Legacy UI Framework

2026 RE-RASSOR Final Design Document

A rigorous audit of the inherited React Native application identified several critical design and functional limitations that necessitate immediate refactoring:

Obsolete Command Mapping:

The legacy UI contains fixed controls mapped exclusively to legacy drum actions ('front_arm_action', 'back_arm_action', 'front_drum_action', 'back_drum_action'). These controls dispatch binary step opcodes (0x05 / 0x06) that are incapable of commanding variable joint angles, holding position, or executing coordinated multi-joint interpolation.

Stylistic Inconsistency & Legacy Graphics:

The commented-out arm component in the legacy code relies on static, untextured 3D CAD renders that clash with the modern, minimalist 2D vector aesthetics of the primary driving dashboard. These 3D assets represent an outdated arm geometry with a controllable wrist and static end effector, directly contradicting our 2026 design (which features a stepper-driven base, BLDC shoulder/elbow joints, a static wrist, and an active paver gripper).

Lack of Visual Position & Telemetry Feedback:

The legacy interface provides no visual representation of physical joint positions, leaving the operator entirely blind to the arm's real-world spatial orientation, current draw, or proximity to mechanical soft/hard limits.

Blocking Network Calls:

Legacy HTTP dispatch methods lack robust retry-handling, timeout fallbacks, and multi-threading isolation, resulting in interface freezing during transient Wi-Fi packet drops.

UI Architectural Redesign & Component Hierarchy

The new UI is built on the existing React Native / Expo framework to maintain cross-platform compatibility on tablets, laptops, and mobile devices. The visual layout is structured as a modular dashboard with tabbed navigation between the 'Rover Drive Mode', 'Arm Manipulator Dashboard' and 'Camera Teleoperation Overlay'.

UI Component Module	React Native Component File	Functional Description & Responsibility
---------------------	-----------------------------	---

2026 RE-RASSOR Final Design Document

Arm Control Dashboard	ArmControlScreen.js	Primary container component hosting joint sliders, preset macro buttons, and status telemetry header.
Joint Slider Array	JointSliderGroup.js	Renders interactive smooth sliders for Base Yaw (0–180°), Shoulder Pitch (0–120°), Elbow Pitch (0–150°), and Gripper Claw.
Vector Kinematic Visualizer	ArmKinematicSVG.js	Dynamic 2D SVG vector component that renders a real-time side-view skeletal representation of arm joint angles.
Camera Overlay Panel	CameraArmOverlay.js	Embeds live ArduCam video stream with interactive crosshairs, ArUco detection boxes, and quick-arm actuation controls.
API Communication Layer	ArmApiDispatcher.js	Encapsulates REST POST payload formatting, SocketIO subscription listeners, and network timeout error handling.

Manipulator Control Architecture

To provide efficient mechanical tuning and operational execution, the redesigned interface implements a dual-mode control architecture:

Preset Macro Command Mode:

Provides one-touch execution buttons for standardized, pre-tested arm configurations:

- [STOW / REST]: Retracts shoulder and elbow to compact resting geometry against chassis plate.
- [GRASP PAVER]: Closes end effector gripper until contact force sensor threshold is reached.

Real-Time Visual Telemetry & 2D Vector Graphic Integration

In order to match the sleek, monochromatic visual language of the existing driving UI (using 2D vector graphic representations of the rover wheels and chassis), the arm interface replaces all legacy 3D renders with a dynamic React Native SVG 2D Vector Visualizer (`ArmKinematicSVG.js``).

The visualizer dynamically computes the forward kinematics from the real-time joint angle feedback provided by the encoders of the Arduino Mega. It provides a clean vector side view of the chassis of the rover arm linkages, joint pivot points, and gripper status. Additionally, could also color-coded status indicators to highlight joint states:

- Green: Joint operating normally within safe angular bounds.
- Yellow: Joint approaching soft angular limits or elevated thermal thresholds.
- Red: Joint fault, limit switch triggered, or communication timeout.

ArduCam Camera Video Integration & ArUco Overlay

The UI features live video feedback from the main ArduCam UC-A53 mounted on the front chassis. When the operator chooses the ‘Camera Teleoperation Overlay’ mode, the video feed is presented in the center of the dashboard, with semi-transparent arm controls overlaid along the edges of the screen.

The camera view also has an ArUco marker detection grid to facilitate future computer vision stretch goals (e.g. autonomous paver recognition). When a standardized 3D paver with an ArUco fiducial marker enters the camera’s field of view, the UI shows a green bounding box around it, estimated distance (in centimeters) and a button to “Lock & Acquire Target” to connect manual teleoperation to the future automated pickup routines.

Safety Ergonomics, Deadman Mechanics, & Emergency Overrides

Global Emergency Stop Banner:

A red [EMERGENCY STOP] banner is rendered persistently in the top-right corner of all UI screens. Clicking this button will send a high-priority HTTP POST to `/api/emergency\_stop` and fire a WebSocket interrupt to immediately disable all motor driver outputs.

Continuous Hold:

Hold mechanics over time. Touch contact is required for joint movement sliders. If the operator removes their finger from a slider, or if the mobile application loses window focus, movement commands immediately revert to velocity=0.0 (holding current position).

Network Watchdog Feedback:

The top status bar shows live round-trip network ping latency (in milliseconds). If communication over Wi-Fi is lost for more than 500 ms, a visual alert 'LINK LOST' is displayed on the UI, and the sending of arm commands is automatically stopped.

Conclusions

Where We Fell Short

Initial Timeline Issues

Our original timeline in Senior Design 1 was a fast-paced style that did not take into consideration the reliance on the lower portions of the rover being completed and the complexity of the design which said parts required. In addition to this miscalculation of each component's time, the schedule was formulated during a calmer period in the team's schedule. Both of these conditions resulted in delays in development. However, after the completion of the base and proper scaling and ratios have been finalized the upper and middle sections of the arm can be developed simultaneously to reduce production time while limiting printing and fitting errors that could potentially waste material.

This mistake is likely due to our team's unfamiliarity with engineering-centered projects and their proper workflow. In a standard computer science workflow, all tasks are treated as parallel objectives in which all sub teams can easily section out material with relatively few dependencies. However, this is not the case for projects such as RE-RASSOR

From this error, we better realized the scope of this project and have a better idea of how to plan and organize the remaining prints for the Version 1 Prototype and the potential Version 2 Prototype. In terms of future allocation, base components crucial to component scaling should be prioritized first; for example, sections like joints and weight bearing moving components. These pieces should likely be handled by the members of the mechanical team with the most time dedicated to the project or the project manager to ensure that they are produced, refined, and finalized quickly to not delay development of the arm overall.

Administration

Budget & Material Procurement

Material procurement for the RE-RASSOR project is managed by FSI rather than operating under a fixed project budget; the team submits requests for required materials and components to the project sponsor for approval and purchase. This procurement process is used for two primary reasons: the availability of existing project resources and sponsor oversight of project expenditures.

At the beginning of the project, the team was granted access to FSI's laboratory located in Research Park. Due to the efforts of previous RE-RASSOR teams, many of the mechanical components, electronics, software resources, and 3D printing equipment required to develop both the robotic arm and its testing rover were already available. As a result, relatively few additional materials needed to be procured throughout the project, with most purchases limited to consumables such as filament and newly implemented hardware.

Additionally, robotics projects often involve expensive mechanical components and electronic hardware, making effective cost management an important consideration. To ensure project funds are used efficiently, the sponsor reviews and approves material requests before purchases are made. This process provides financial oversight while allowing the team to obtain the necessary components to complete the project.

Current Cost Breakdown

Under the guidance of our sponsor, we have estimated that this arm will cost around 600 USD for the components and mechanisms this team constructs over the course of this project. Therefore, we plan to use this figure as a budget for our cost tables below. This would make the estimated total cost of the

2026 RE-RASSOR Final Design Document

rover 1,800 USD for recreation from scratch by other institutions or third parties who may wish to use our research as defined in our impacts section.

Arm Cost Table

Material / Component	Quantity	Estimated Unit Cost	Estimated Value	2026 Team Expenditure
PETG Filament (Final Components)	2 Rolls	\$25.00	\$50.00	\$0.00
PLA Filament (Prototype Components)	1 Roll	\$20.00	\$20.00	\$0.00
M3 Fasteners (Screws, Nuts, Washers)	Assorted	\$15.00	\$15.00	\$0.00
M4 Fasteners (Mounting Hardware)	Assorted	\$15.00	\$15.00	\$0.00
Totals			\$100.00	\$0.00

Table 3: Arm Material Costs

Rover Body Cost Table

Material / Component	Quantity	Estimated Unit Cost	Estimated Value	Team Expenditure
Bambu Lab P1P	1	\$699.99	\$699.99	\$0.00
Libre Computer (Potato SBC)	1	\$40.00	\$40.00	\$0.00
Arduino Mega 2560	1	\$22.99	\$22.99	\$0.00
RAMPS Control Board	1	\$9.39	\$9.39	\$0.00
Stepper Motor Drivers	1 Set	\$10.19	\$10.19	\$0.00
Wi-Fi Adapter	1	\$10.00	\$10.00	\$0.00
32 GB Micro SD Card	1	\$8.99	\$8.99	\$0.00
Stepper Motors	4	\$14.99	\$14.99	\$0.00
Battery Charger	1	\$35.99	\$35.99	\$0.00
Emergency Stop Button	1	\$11.99	\$11.99	\$0.00
Camera	1	\$34.99	\$34.99	\$0.00

2026 RE-RASSOR Final Design Document

Battery	1	\$39.99	\$39.99	\$0.00
Buck Converter	1	\$7.94	\$7.94	\$0.00
Power Splitter	1	\$6.48	\$6.48	\$0.00
USB Cable	1	\$8.99	\$8.99	\$0.00
SD Card Reader	1	\$11.99	\$11.99	\$0.00
Polymaker PETG Filament	5 Rolls	\$22.99/Roll	\$114.95	\$0.00
Male/Female Pigtail Connectors	1 Set	\$9.49	\$9.49	\$0.00
Heat Shrink Spade Connectors	1 Set	\$16.99	\$16.99	\$0.00
Heat Shrink Fork Connectors	1 Set	\$9.99	\$9.99	\$0.00
12V Cooling Fan	1	\$12.99	\$12.99	\$0.00
Wire Crimping Tool	1	\$8.99	\$8.99	\$0.00
SAE Power Port	1	\$11.99	\$11.99	\$0.00
Totals			\$1,275.14	\$0.00

Table 4: [1][5], [9] Rover Body Material Costs

Printing Time Breakdown

Our current Bambu Studio estimates show that the arm base will require approximately 2 hours to print and will use 61.07 grams of PETG. The mounting platform is considerably larger, with an estimated print time of 12 hours and 47 minutes and a material requirement of 373.47 grams. Together, these two completed designs account for 14 hours and 47 minutes of printing and 434.54 grams of PETG.

Most of the remaining arm components are expected to have print times closer to the 2-hour estimate of the arm base. The larger structural portion of the arm may require an amount of PETG similar to the mounting platform. The gripper and its control components will likely use an amount closer to the arm base. Note that several gripper parts may be arranged on the same print bed when space allows, which could reduce the number of separate print sessions.

Based on the current models, the complete final assembly may require approximately 864 grams of PETG and close to 30 hours of printer operation. We estimate that at least two 800 gram rolls should be available for the final build. The additional material provides room for revised parts, failed prints,

2026 RE-RASSOR Final Design Document

support material, and small components that have not yet been finalized. These figures may change as the remaining CAD models are completed and imported into Bambu Studio.

Assembly	Estimated Print Time	Estimated PETG Use
Arm Base	2 hr.	61.07 g
Mounting Platform	12 hr. 47 min	373.47 g
Remaining Arm Structure	Approx. 12 hr.	Approx. 370 g
Gripper & Control Components	Approx. 2 hr.	Approx. 60 g
Estimated Total	Approx. 28 hr. 47 min	Approx. 864.54 g

User Stories

The following user stories define the primary functional requirements of the RE-RASSOR robotic arm from the perspective of the system's intended stakeholders. These stories were used to guide design decisions and ensure the completed system addresses the needs of all parties with a stake or interest in this project

Rover Operator from a User Perspective

As a rover operator, I want to remotely control the robotic arm through the rover's web-based user interface so that I can safely manipulate objects and perform tasks without physically interacting with the rover.

This functionality allows an operator to issue movement commands through the existing HTTP-based control interface while simultaneously monitoring the rover's surroundings using the onboard camera system. The robotic arm should respond accurately and consistently to user inputs, enabling reliable operation during testing and future research activities.

Rover Operator from a System Administrator Perspective

As a rover operator, I want the rover to communicate through an isolated wireless network so that the robotic arm can be controlled reliably without being affected by institutional network restrictions.

The rover should establish communication through a dedicated wireless router, allowing the onboard computer and operator interface to exchange HTTP requests without relying on university-managed network infrastructure. This improves communication reliability while simplifying deployment in different testing environments.

Future RE-RASSOR Development Team

As a future RE-RASSOR development team, I want the robotic arm's hardware, software, and documentation to be well organized so that I can continue development without redesigning or reverse engineering existing work.

Because the RE-RASSOR project is designed to be inherited by future senior design teams, comprehensive documentation, organized source code, CAD models, and clearly defined hardware interfaces are essential. This enables future teams to focus on expanding system capabilities rather than reconstructing previously completed work.

FSI Sponsor

As the Florida Space Institute sponsor, I want the robotic arm to integrate seamlessly with the existing RE-RASSOR platform so that it expands the rover's capabilities while remaining compatible with previously developed hardware and software.

The robotic arm should interface with the existing control architecture, electrical systems, and software infrastructure without requiring extensive modifications to the base rover. Maintaining compatibility preserves previous development efforts while allowing future enhancements to be implemented more efficiently.

Development Team Member

As a member of the development team, I want the robotic arm to utilize commercially available components and modular mechanical assemblies so that the system is easier to manufacture, maintain, repair, and upgrade.

Selecting standardized hardware, readily available bearings, fasteners, electrical connectors, and 3D-printed modular components simplifies manufacturing and assembly while reducing maintenance costs. A modular design also allows damaged components to be replaced individually without requiring the reconstruction of the entire robotic arm.

Researcher

As a researcher, I want the robotic arm to manipulate objects accurately and repeatedly so that experiments conducted with the RE-RASSOR platform produce consistent and reliable results.

The robotic arm should provide predictable movement, stable positioning, and repeatable operation when performing manipulation tasks. Reliable operation improves the quality of experimental data and increases confidence in the rover's ability to perform future research objectives.

Summary of Current Completed Tasks and Subtasks

In our efforts towards this project so far, we have made significant progress towards reaching the project goals. This progress is measurable by the various tasks and subtasks we have completed thus far. The number of subtasks we have completed is too many to enumerate here, but a summary of some of our efforts will be listed.

We have established multiple ways to meet and communicate with each other and the sponsor. We communicate constantly through our team Discord, discuss tasks, and share resources and documents. We hold weekly meetings in the Florida Space Institute computer lab, the same place where the rover, the printers, and other relevant materials are held. We also meet the sponsor at that location occasionally, as well as communicating with the sponsor over email.

We have examined, tested, and verified the existing products and components provided by previous teams. We analyzed the previous teams' code to ensure that we have all of the components and that none are nonfunctional or outdated. We checked the existing rover prototype for any obvious damage or wear and checked that all electrical components were present in the expected locations and connected correctly. We tested that the existing control interface was functional and able to be used from the available devices. We were then able to proceed to testing the functionality of all of the existing parts together. In this process, we established a baseline for measuring our own modifications and products against, and an understanding of what needs to be altered or maintained.

Many of the materials and tools needed for the more generic tasks related to the project were already obtained by previous teams working on this project and can be reused easily. We were able to perform an inventory of these materials and tools to determine which additional materials we require to complete our tasks. Due to our communication set up, we were able to request these additional materials from the sponsor over email, including specific and/or non-reusable materials.

We have created and tested designs for multiple components that either attach to or are part of the paver arm. This includes both the mounting platform and the arm base. These components went through drafts, designs, testing, and adjustments to reach their current forms. More information on how these components were modified is available in the [Component Modifications Made During Development](#) section of this document,

We also have made significant progress on the other components of the paver arm. The end effector, joint mechanisms, and structural connectors are all modeled, and prototypes of these components will be printed soon for testing. After the prototypes are tested and any necessary modifications have been made and re-tested, the final designs will be completed and printed.

We have also made plans for how we will integrate our UI updates into the existing UI. Based on our analysis of the existing UI, we have a list of choices that were made during the creation of that design, that we will likely mimic to allow our UI to fit in better. We also have determined exactly which elements we will need to add to the UI to make it functional and have drafted parts of the code needed for those elements.

We have completed drafts of the HTTP messages that will be needed to communicate the instructions to paver arm from the user's controls. We also have an outline of the code that will be used by the Arduino microcontroller to control the BLDC motors in the paver arm. Both only require minimal polishing before they can be tested, and any issues that arise can be fixed.

In order to organize both the completed and future tasks we have constructed through the design process of our rover; we have created a Jira board following the Kanban template. This is described in more detail in the following section.

Jira Workflow

Our workflow is organized around the Kanban method as an Agile framework, as required by our project. This is being used as an alternative to the Scrum method that most other senior design teams are using. Because we are using the Kanban method, our Jira board therefore uses the free Kanban template provided by Atlassian. [38] This results in our Jira board being organized into tasks in seven phases, as shown below.

2026 RE-RASSOR Final Design Document

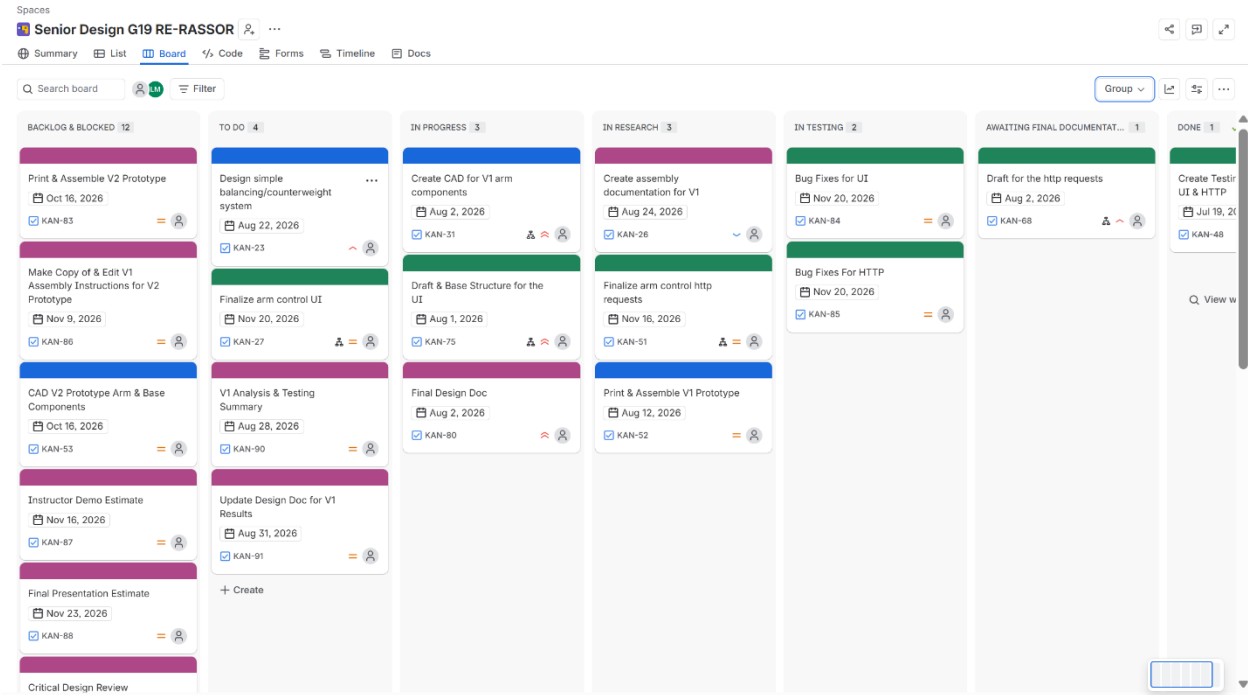


Figure 24: Our Team's Jira Board as of July 25th, 2026

The seven phases we use are “Backlog and Blocked”, “To Do”, “In Progress”, “In Research”, “In Testing”, “Awaiting Final Documentation”, and “Done”. The explanations of each phase are as follows:

- Backlog and Blocked – Tasks that are temporarily on hold, due to lack of resources, prerequisites, or other requirements to work on.
- To Do – Tasks that will be worked on immediately once other tasks are complete.
- In Progress – Tasks currently being worked on.
- In Research – Tasks that are being researched before beginning or resuming work.
- In Testing – Tasks that are believed to be completed but require further review or testing to verify results.
- Awaiting Final Documentation – Tasks that are completed but require additional documentation to be written about them for one or more reasons.
- Done – Tasks that are completed and require no further work.

2026 RE-RASSOR Final Design Document

All of the tasks on the Jira board are also marked with the deadlines they should be completed by. Individuals are assigned tasks as required. Some tasks are assigned to the entire team, or to either the front end or back end sub-teams.

Some tasks are divided into subtasks; this is done for the purposes of more granular tracking or assignment. The subtasks can be individually marked as different phases, separately from the larger task itself.

Acknowledgements

Project Guidance

There are several individuals who made this project possible. As such, the 2026 RE-RASSOR team would like to individually thank these members of our community at UCF and FSI for their aid and guidance as we have worked on this project.

The team would like to express its sincere gratitude to **Mr. Christian Vincent** of the Florida Space Institute for serving as the project's sponsor representative. His technical guidance, project feedback, and continued support have been invaluable throughout the design and development process.

The team would also like to thank **Dr. Gerber**, Professor of Senior Design at the University of Central Florida, for his mentorship, project oversight, and constructive feedback throughout the course.

Lastly, this team would like to thank the **2025 BLDC RE-RASSOR Team** who took time out of their busy schedules at the beginning of this project to pass down valuable resources and advice that would not have been accessible otherwise

Institutional Resources

In addition to the individuals who supported this project, the team would like to acknowledge the organizations whose facilities and resources made the development process possible.

A significant portion of this project's development was conducted within the facilities provided by the **Florida Space Institute (FSI)**. Access to the laboratories in **Research Building 1** and the **Exolith Lab**, previous RE-RASSOR prototypes from various stages in development from the past teams, mechanical components for testing and design validation, and additive manufacturing equipment greatly accelerated the team's development process.

In particular, the dedicated 3D printers provided by FSI enabled rapid prototyping and iterative design refinement, allowing mechanical components to be manufactured, evaluated, and improved throughout the project with minimal cost to the team. Access to these facilities and resources substantially reduced development time and made repeated design iterations feasible.

The team would also like to acknowledge the **Stephen Hawking Center for Microgravity Research** for providing financial support throughout the project's research and development period. This

funding enabled the procurement of consumable materials, including 3D printing filament and mechanical components required for the fabrication and assembly of the rover arm.

Appendix

Many of the references cited below include internal engineering documentation, GitHub repositories, hardware datasheets, and previous RE-RASSOR Senior Design reports. These resources served as the technical foundation upon which the 2026 RE-RASSOR Arm Development project was designed and implemented. Our sources and reference are as follows:

Previous RE-RASSOR Projects

[1] Florida Space Institute, "2022 RE-RASSOR Arm Development Final Design Document," Google Drive, [Online], Available: [Arm Team Resources - Google Drive](#)

[2] Florida Space Institute, "2023 RE-RASSOR Extension Final Design Document," Orlando, FL, USA, 2023.

[3] Florida Space Institute, "2025 RE-RASSOR Brushless DC Motor Control Final Design Document," Orlando, FL, USA, 2025.

[4] Florida Space Institute, "2023 RE-RASSOR Extension GitHub Repository," GitHub repository. [Online]. Available:

<https://github.com/FlaSpaceInst/2023-RE-RASSOR-Extension>

[5] Florida Space Institute, "2025 RE-RASSOR BLDC Motor Control GitHub Repository," GitHub repository. [Online]. Available: <https://github.com/FlaSpaceInst/2025-Fall-UCF-BLDC-Motor-Control>

Internal Documentation

[6] Florida Space Institute, "RE-RASSOR Project Requirements," Internal Project Documentation, Orlando, FL, USA, 2026.

[7] Florida Space Institute, "Sponsor Meeting Notes," Internal Documentation, Orlando, FL, USA, 2026.

[8] Florida Space Institute, "Existing RE-RASSOR CAD Assemblies," Internal Engineering Documentation.

[9] Florida Space Institute, "Electrical Wiring and Hardware Documentation," Internal Engineering Documentation.

Hardware Documentation

[10] Raspberry Pi Foundation, "Raspberry Pi 4 Documentation." [Online]. Available:

<https://www.raspberrypi.com/documentation/>

[11] Arduino, "Arduino Mega Documentation." [Online]. Available:

<https://docs.arduino.cc/>

[12] ArduCam, "ArduCam Camera Documentation." [Online]. Available:

<https://docs.arducam.com/>

[13] Bambu Lab, "Bambu Lab X1 Carbon User Manual." [Online]. Available:

<https://wiki.bambulab.com/>

[14] Bambu Lab, "Bambu Studio Documentation." [Online]. Available:

<https://wiki.bambulab.com/>

[15] Autodesk, "Fusion 360 Documentation." [Online]. Available: <https://help.autodesk.com/>

[16] Autodesk, "AutoCAD Documentation." [Online]. Available:

<https://help.autodesk.com/>

[17] Onshape Inc., "Onshape Documentation." [Online]. Available:

<https://cad.onshape.com/help/>

Software Documentation

[18] Meta Platforms, Inc., "React Native Documentation." [Online]. Available:

<https://reactnative.dev/>

[19] Expo, "Expo Documentation." [Online]. Available:

<https://docs.expo.dev/>

[20] Git Documentation Project, "Git Documentation." [Online]. Available:

<https://git-sem.com/doc>

[21] GitHub, "GitHub Documentation." [Online]. Available:

<https://docs.github.com/>

[22] Microsoft, "Visual Studio Code Documentation." [Online]. Available:

<https://code.visualstudio.com/docs>

[23] Arduino, "Arduino IDE Documentation." [Online]. Available:

<https://docs.arduino.cc/software/ide/>

[24] Open Robotics, "ROS 2 Documentation." [Online]. Available:

<https://docs.ros.org/>

[25] Gazebo, "Gazebo Simulator Documentation." [Online]. Available:

<https://gazebosim.org/docs>

Design Inspiration

[26] daGHIZmo, "EEZYbotARM Open Source Robotic Arm," GitHub repository. [Online]. Available:

<https://github.com/daGHIZmo/EEZYbotARM>

[27] daGHIZmo, "EEZYbotARM Assembly Instructions." [Online]. Available:

<https://www.eezyrobots.it/>

Robotics & Technical References

[28] I. Asimov, I, Robot. New York, NY, USA: Gnome Press, 1950.

[29] NASA, "NASA Technical Reports Server (NTRS)." [Online]. Available:

<https://ntrs.nasa.gov/>

[30] NASA, "Kennedy Space Center." [Online]. Available:

<https://www.nasa.gov/kennedy/>

Manufacturer Datasheets

[31] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet."

[32] Arduino, "Arduino Mega 2560 Datasheet."

[33] Nermak, "12.8V LiFePO4 Battery Specifications."

[34] ArduCam, "UC-A53 Camera Datasheet."

[35] Netgear, "Nighthawk Router User Guide."

[36] Manufacturer documentation for BLDC motors utilized by the RE-RASSOR platform.

[37] Manufacturer documentation for stepper motor utilized in the arm base assembly.

Project Methodology

[38] D. Radigan, "What is Kanban in project management? Breaking down the methodology."

Atlassian. Available: <https://www.atlassian.com/agile/kanban> (Accessed Jul. 31, 2026)